

POLITECNICO DI TORINO

CORSO DI PROGRAMMAZIONE DI SISTEMA / API PROGRAMMING — A.A. 2025/2026



GEORUGGINE

Manuale del progettista

Relazione tecnica finale di architettura software e specifica di dettaglio

Specifiche tecniche di progetto e architettura di sistema

Piattaforma distribuita di telemetria, calcolo geodetico e messaggistica in tempo reale

Autore del progetto

Nicola Beccaceci

nicola.beccaceci@gmail.com

Indice

1	Introduzione, visione di sistema e requisiti	1
1.1	Visione di sistema e scenario operativo	1
1.2	Modello degli stakeholder e user personas	2
1.3	Requisiti funzionali di sistema	3
1.4	Requisiti non funzionali e attributi qualitativi del sistema	7
2	Architettura generale e modello a blocchi	9
2.1	Scomposizione in crate e confini di responsabilità	9
2.2	Architettura di contesto C4, modello di comunicazione e deployment	10
2.3	Schede delle decisioni architetturali fondamentali (ADR-001 — ADR-007)	13
3	Modello dei dati e invarianti di dominio	17
3.1	Censimento completo delle strutture dati, enumerazioni e tratti	17
3.2	Serializzazione e deserializzazione Serde	23
3.3	Invarianti di dominio e regole di validazione	24
3.4	Verifica empirica e matrice dei test del modello dati	25
4	Protocollo di rete wire e comunicazione	26
4.1	Meccanismo di framing JSON-lines su stream TCP	26
4.2	Ingestione asincrona bufferizzata e multiplexing	26
4.3	Diagrammi di sequenza del protocollo di comunicazione	29
4.4	Matrice completa di gestione degli errori (ServerError) e del trasporto	32
4.5	Verifica empirica e matrice dei test del protocollo di rete	34
5	Macchina a stati finiti (FSM) del veicolo e dinamica temporale	35
5.1	Formalizzazione matematica dell'automa a stati	35
5.2	Logica di transizione e condizioni di guardia	35
5.3	Dinamica temporale: soglia di stasi (180s) e cadenza telemetrica (30s)	36
5.4	Diagramma di stato e timeline temporale	36
5.5	Architettura del terminale CLI e gestione interattiva dell'input (Crossterm)	38
5.6	Verifica empirica e matrice dei test della FSM e del terminale	44
6	Algoritmi geospaziali, telemetria e aggregazione temporale	45
6.1	Filtraggio del rumore strumentale GPS e calcolo della velocità media	45
6.2	Finestre temporali di calendario UTC	46

6.3	Algoritmo di scansione a singolo passaggio $\mathcal{O}(\log N + K)$	46
6.4	Multiplexing asincrono del terminale nel loop <code>tokio::select!</code>	47
6.5	Verifica empirica e matrice dei test degli algoritmi telemetrici	53
7	Motore di simulazione cinematica	54
7.1	Pattern strategy per i generatori di coordinate (<code>PositionProvider</code>)	54
7.2	Fornitore coordinate da file CSV (<code>FilePositionProvider</code>)	54
7.3	Verifica empirica e matrice dei test della simulazione cinematica	57
8	Topologia di concorrenza e gestione della memoria	58
8.1	Architettura di concorrenza del server multi-tenant	58
8.2	Modello di concorrenza del client: Event loop reattivo single-owner	58
8.3	Dimostrazione formale dell'assenza di data race in Rust	59
8.4	Isolamento dell'I/O bloccante e integrità della persistenza SQLite	59
8.5	Diagrammi di topologia di concorrenza e mappa di memoria	60
8.6	Verifica empirica e matrice dei test di concorrenza e sincronizzazione	62
9	Analisi comparativa delle scelte architetturali	63
9.1	Memoria condivisa (<code>Arc<RwLock></code>) vs pattern ad attori (<code>mpsc</code>)	63
9.2	Thread nativo OS 1:1 vs runtime asincrono Tokio M:N	63
9.3	Primitive atomiche vs lock pesanti <code>Mutex/RwLock</code>	64
9.4	Matrice comparativa completa dei 4 modelli a 8 criteri	65
10	Requisiti non funzionali, monitoraggio risorse e guida di build	67
10.1	Modulo di monitoraggio CPU 120s e istogramma ASCII	67
10.2	Ottimizzazione estrema del footprint del binario eseguibile	67
10.3	Supporto multi-piattaforma e matrice dei target triples LLVM	68
10.4	Tassonomia della suite di collaudo e test End-to-End (180 test)	69
10.5	Guida operativa per lo sviluppatore (build, test, lint, run)	70
11	Conclusioni ed evoluzioni future	74
11.1	Sintesi dei risultati raggiunti	74
11.2	Possibili evoluzioni future	74
A	Schemi JSON del protocollo wire	75
B	Matrice di tracciabilità requisiti-test	76
B.1	Tracciabilità dei requisiti funzionali (RF-01 — RF-06)	76
B.2	Tracciabilità dei moduli architetturali avanzati e non funzionali (RNF)	79

Capitolo 1

Introduzione, visione di sistema e requisiti

1.1 Visione di sistema e scenario operativo

Il progetto **Georuggine** nasce per rispondere all'esigenza industriale di disporre di un'infrastruttura di telemetria e gestione operativa per flotte di veicoli commerciali pesanti e leggeri, caratterizzata da massima efficienza computazionale, footprint di memoria ridotto e affidabilità deterministica.

Nei moderni sistemi di trasporto intelligenti, i terminali installati a bordo dei veicoli devono operare continuamente in contesti connettivi eterogenei, soggetti a latenze variabili su reti cellulari, frequenti disconnessioni fisiche e vincoli energetici e di memoria. Parallelamente, la centrale operativa di flotta deve essere in grado di aggregare in tempo reale i dati posizionali di centinaia o migliaia di veicoli connessi, calcolare metriche statistiche aggregate, quali distanze chilometriche, velocità effettive di crociera, tempi di guida e pause di riposo nel rispetto delle normative vigenti, e recapitare direttive operative istantanee, come comunicazioni broadcast e messaggi diretti.

Georuggine implementa un'architettura distribuita **Client/Server** asincrona, interamente sviluppata in **Rust**, che garantisce l'assenza assoluta di *data race* a tempo di compilazione, un'elevatissima scalabilità sull'I/O di rete, e una persistenza transazionale ACID locale basata su un motore SQLite embedded completamente disaccoppiato dall'event loop asincrono.

1.2 Modello degli stakeholder e user personas

La progettazione del sistema è stata guidata dall'analisi dei requisiti e dei vincoli operativi degli attori riportati in Tabella 1.1.

Stakeholder	Ruolo operativo	Obiettivi primari e responsabilità	Canale d'interfaccia
Fleet Manager Centrale Logistica	Amministratore della flotta	Supervisione real-time dello stato flotta (InMovimento , Fermo , Sconnesso); interrogazione metriche su finestre temporali; invio ordini (broadcast e direct); monitoraggio CPU del server.	Shell CLI interattiva su <code>stdin</code> server (<code>list</code> , <code>stats</code> , <code>broadcast</code> , <code>direct</code> , <code>cpu</code> , <code>chart</code>).
Autista / Driver Terminale di Bordo	Conducente veicolo	Autenticazione con credenziali protette; trasmissione telemetria GPS ogni 30s; ricezione ordini di servizio ed emissione conferme testuali.	Terminale CLI di autenticazione e task asincrono di fondo.
Ingegnere software Sviluppo e test	Sviluppatore di sistema	Estensione con nuove strategie cinematiche (Pattern Strategy); integrazione endpoint telemetrici; conformità a 100% Safe Rust e suite di test.	Cargo toolchain, libreria di dominio georuggine , suite automatizzata.
DevOps / SRE	Amministratore infra	Distribuzione binari esecutivi autonomi compatti (< 2 MB) senza dipendenze C esterne; alta disponibilità e basso consumo risorse.	Binari Release, file database georuggine.db , log <code>cpu_usage.log</code> .
Commissione d'Esame	Valutatore accademico	Verifica del rigore formale architetturale, assenza matematica di data race, correttezza Haversine e analisi comparativa concorrenza.	Manuale del Progettista, sorgenti Safe Rust, 180 test automatizzati.

Tabella 1.1: Matrice degli stakeholder, ruoli operativi e canali d'interfaccia.

1.3 Requisiti funzionali di sistema

Il capitolato tecnico di Georuggine è strutturato in sei macro-requisiti funzionali, ripartiti gerarchicamente in requisiti principali (1., 2., 3., ...) e relativi sotto-requisiti operativi (1.1, 1.2, ...). La Tabella 1.2 sintetizza l'intera matrice dei requisiti con le rispettive specifiche tecniche e criteri di verifica.

Tabella 1.2: Matrice gerarchica dei requisiti funzionali di sistema.

ID	Requisito	Specifica tecnica e criteri di verifica
1. Gestione dell'autenticazione, registrazione e sessioni concorrenti		
1.1	Validazione username	Lunghezza compresa tra 4 e 20 caratteri, esclusivamente alfanumerici ASCII ([a-zA-Z0-9]). Stringhe non conformi sono respinte con <code>ServerError::InvalidUsernameLength</code> o <code>ServerError::InvalidUsernameCharacters</code> .
1.2	Complessità password	Lunghezza minima di 8 caratteri con presenza obbligatoria di almeno una lettera minuscola ([a-z]), una maiuscola ([A-Z]) e una cifra numerica ([0-9]). Violazioni causano il rigetto con <code>ServerError::PasswordTooShort</code> o <code>ServerError::PasswordMissingRequiredCharacters</code> .
1.3	Hashing crittografico	Cifratura password tramite algoritmo Bcrypt con fattore di costo standard $2^{12} = 4096$ iterazioni (<code>DEFAULT_COST = 12</code>) e salatura automatica a 128 bit. Le credenziali in chiaro non vengono mai persistite su disco né mantenute in RAM oltre la fase di verifica.
1.4	Persistenza e unicità	I profili utente sono archiviati nella tabella relazionale <code>users</code> del database SQLite embedded (<code>georuggine.db</code>). La registrazione di uno username già presente viene respinta atomicamente con <code>ServerError::UsernameTaken</code> .
1.5	Mutua esclusione sessione	Un utente autenticato non può stabilire contemporaneamente una seconda sessione attiva. Un tentativo di login simultaneo su un account già connesso viene respinto con <code>ServerError::AlreadyLoggedIn</code> , preservando la sessione preesistente.
1.6	Cleanup deterministico	Alla chiusura dello stream TCP da parte del client (EOF, <code>bytes == 0</code> o caduta socket), il server dealloca la mailbox MPSC dalla mappa <code>active_clients</code> e commuta lo stato del veicolo a <code>UserState::Disconnected</code> .

Continua alla pagina successiva...

Tabella 1.2 (continua dalla pagina precedente)

ID	Requisito	Specifica tecnica e criteri di verifica
2. Trasmissione telemetrica periodica e validazione coordinate		
2.1	Campionamento temporizzato	Il client di bordo trasmette la propria posizione geografica con cadenza deterministica fissa di 30 secondi esatti, scandita mediante <code>tokio::time::interval</code> .
2.2	Struttura posizione GPS	Pacchetto telemetrico incapsulato nella struct <code>Position</code> , contenente latitudine ($\phi \in [-90.0, +90.0]^\circ$), longitudine ($\lambda \in [-180.0, +180.0]^\circ$) su datum geodetico WGS-84 e timestamp Unix in secondi UTC.
2.3	Validazione e monotonicità temporale	Il server valida per ciascun utente la monotonicità temporale $t_{\text{new}} \geq t_{\text{last}}$, dove t_{last} rappresenta il timestamp dell'ultima coordinata registrata; campioni retroattivi sono scartati con <code>ServerError::OutOfOrderPosition</code> . Il calcolo geodetico adotta clamp numerico sull'argomento dell'Haversine per prevenire instabilità floating-point su coordinate reali.
3. Macchina a stati finiti (FSM) del monitoraggio veicolo		
3.1	Spazio stati tipizzato	Stato cinematico modellato dall'enum <code>UserState</code> : <code>Disconnected</code> (assenza di connessione di rete), <code>Stopped(u64)</code> (veicolo connesso e fermo, con timestamp di ingresso nella sosta), <code>Moving(u64)</code> (veicolo connesso e in marcia, con timestamp dell'ultimo movimento).
3.2	Transizione su stati ≥ 180 s	Se un veicolo nello stato <code>Moving(t_m)</code> invia coordinate identiche per un intervallo $\Delta t = (t_{\text{new}} - t_m) \geq 180$ secondi (3 minuti), il server commuta automaticamente lo stato in <code>Stopped(t_{new})</code> . Se $\Delta t < 180$ s, lo stato permane <code>Moving(t_m)</code> .
3.3	Transizione a movimento	Alla ricezione di una coordinata con variazione spaziale rispetto alla precedente, il veicolo in stato <code>Stopped</code> commuta istantaneamente allo stato <code>Moving(t_{new})</code> .
4. Motore di simulazione cinematica (Pattern Strategy)		
4.1	Trait polimorfo	Astrazione della cinematica veicolare dietro il trait <code>PositionProvider</code> , permettendo la selezione modulare del modello di moto senza alterare il loop di rete.

Continua alla pagina successiva...

Tabella 1.2 (continua dalla pagina precedente)

ID	Requisito	Specifica tecnica e criteri di verifica
4.2	Strategia FilePositionProvider	Riproduzione sequenziale e temporizzata di coordinate GPS da traccia CSV (<code>latitude, longitude</code>) con sorgente temporale iniettabile. All'End-Of-File (EOF), la strategia entra in modalità <i>latch</i> , continuando a fornire l'ultima posizione valida con timestamp aggiornato all'ora corrente per indurre la stasi FSM dopo 180s.
4.3	Estendibilità architetturale	Predisposizione per l'integrazione di generatori sintetici o sorgenti live conformi al trait <code>PositionProvider</code> , preservando il disaccoppiamento con il runtime di trasmissione.
5. Modulo di analisi geospaziale e finestre temporali di calendario		
5.1	Distanza ortodromica	Calcolo delle distanze reali tramite la formula dell'Haversine ($R_{\text{Earth}} = 6371.0$ km) stabilizzata contro anomalie di arrotondamento IEEE 754 mediante clamp $[0.0, 1.0]$.
5.2	Filtraggio jitter GPS	Riconoscimento e assorbimento delle fluttuazioni strumentali GPS inferiori o pari a $\epsilon_{\text{GPS}} = 1.0$ m (0.001 km), classificate come soste anziché marcia attiva.
5.3	Velocità media effettiva	Determinazione della velocità media $v_{\text{avg}} = \frac{D_{\text{tot}}}{T_{\text{mov}}/3600}$ [km/h] computata rigorosamente sui soli intervalli temporali in movimento (T_{mov}), con esclusione delle pause.
5.4	Finestre temporali UTC	Partizionamento temporale calendariale deterministico: <code>CurrentDay</code> (mezzanotte 00:00:00 UTC), <code>CurrentWeek</code> (Lunedì 00:00:00 UTC ISO-8601) e <code>CurrentMonth</code> (giorno 1 ore 00:00:00 UTC).
5.5	Scansione temporale logaritmica	Algoritmo $\mathcal{O}(\log N + K)$ basato su ricerca binaria <code>partition_point</code> e scansione lineare a ritroso a finestre temporali adiacenti con integrazione della coda temporale attiva.
6. Canale di messaggistica e diagnostica risorse		
6.1	Messaggistica centrale → autisti	Supporto per dispacci broadcast a tutti i veicoli connessi e comunicazioni dirette mirate (<i>unicast</i>) verso singoli driver autenticati.
6.2	Messaggistica autista → centrale	Canale asincrono per l'invio di notifiche testuali dal veicolo verso la console della centrale logistica con formattazione protetta da sovrascritture.

Continua alla pagina successiva...

Tabella 1.2 (continua dalla pagina precedente)

ID	Requisito	Specifica tecnica e criteri di verifica
6.3	Mailbox bounded e backpressure	Code MPSC con buffer limitato a 32 messaggi per ciascun client. L'invio non bloccante via <code>try_send</code> previene il collasso o lo stallo del server a fronte di client lenti, sollevando <code>ServerError::MailboxFull</code> .
6.4	Logging periodico CPU 120s	Background task che campiona il consumo percentuale CPU del processo server ogni 120 secondi tramite <code>sysinfo</code> , persistendo i campioni su <code>cpu_usage.log</code> e conservando un buffer circolare di 10 elementi (20 minuti).
6.5	Istogramma ASCII	Rappresentazione grafica a barre su 20 colonne visualizzabile su richiesta dalla shell amministrativa, corredata di warning visivo per carichi $\geq 80\%$.

1.4 Requisiti non funzionali e attributi qualitativi del sistema

In Tabella 1.3 vengono formalizzati i requisiti non funzionali di sistema e le relative metriche quantitative di accettazione.

ID	Nome requisito	Specifica tecnica e obiettivo quantitativo
RNF-01	Memory safety e zero data race	Assenza assoluta di <i>undefined behavior</i> , <i>memory leaks</i> e <i>data race</i> , verificata a tempo di compilazione tramite 100% Safe Rust (<code>#![forbid(unsafe_code)]</code> implicito nella logica applicativa).
RNF-02	Scalabilità e bassa latenza I/O	Modello asincrono basato su Tokio con multiplexing I/O ad alte prestazioni. Latenza di elaborazione e routing del singolo frame < 1 ms; supporto per migliaia di connessioni concorrenti con memoria inferiore a 1 MB per 1000 task.
RNF-03	Minimizzazione binary footprint	Profilo di rilascio ultra-ottimizzato (<code>opt-level = "z"</code> , <code>fat LTO</code> , <code>codegen-units = 1</code> , <code>panic = "abort"</code> , <code>strip = true</code>) che garantisce una dimensione dell'eseguibile server inferiore a 2.0 MB (valore ottenuto: 1.35 MB , -96.4%).
RNF-04	Integrità e transazionalità ACID	Persistenza affidabile su SQLite embedded tramite transazioni atomiche isolate in <code>tokio::task::spawn_blocking</code> , proteggendo il file <code>georuggine.db</code> da corruzioni in caso di arresto improvviso del processo.
RNF-05	Portabilità cross-platform	Compatibilità nativa per architetture Tier 1 e Tier 2: Linux (glibc/musl), macOS (ARM64/x86_64), Windows (MSVC), Android (ARM64) e iOS, grazie all'embedding statico dei sorgenti C di SQLite senza dipendenze esterne <code>.so/.dll</code> .
RNF-06	Robustezza e zero-panic wire protocol	Il server non deve mai andare in panico a causa di payload malformati, caratteri non validi o disconnessioni impreviste: tutti gli errori sono gestiti tramite il tipo algebrico <code>Result<T, ServerError></code> .

Tabella 1.3: Matrice dei requisiti non funzionali (RNF) di sistema.

A integrazione dei requisiti non funzionali, la Tabella 1.4 sintetizza gli attributi qualitativi intrinseci e le relative scelte implementative adottate nell'architettura di Georuggine.

Attributo qualitativo	Implementazione architetturale nel sistema Georuggine
Idoneità funzionale	Copertura integrale del 100% dei requisiti RF-01–RF-06; calcolo geodetico esatto di Haversine e aggregazione calendariale a singolo passaggio.
Efficienza prestazionale	Multiplexing I/O asincrono non bloccante con Tokio (M:N); profilazione LTO con footprint di soli 1.35 MB; overhead CPU < 0.1% a riposo.
Compatibilità	Wire protocol basato su JSON-lines UTF-8 standard (RFC 8259); coordinate geodetiche conformi allo standard internazionale WGS-84.
Usabilità	Console shell amministrativa interattiva con visualizzazione istogrammi ASCII a 20 colonne e avvisi visivi di carico elevato ([HIGH LOAD]).
Affidabilità e tolleranza ai guasti	Crash-freedom garantita da Safe Rust; FSM deterministica con gestione della stasi e disconnessioni pulite (EOF); isolamento I/O SQLite.
Sicurezza	Crittografia delle credenziali tramite Bcrypt a costo 2^{12} ; mutua esclusione delle sessioni attive; query SQL con prepared statements (anti-SQL Injection).
Manutenibilità	Scomposizione modulare con crate di dominio agnostico (georuggine); pattern Strategy polimorfo; suite esaustiva di 180 test automatizzati.
Portabilità	Embedding statico di SQLite C; zero dipendenze dinamiche C di terze parti; target triples verificati per Linux musl, macOS, Windows e Android.

Tabella 1.4: Attributi qualitativi intrinseci e garanzie architetturali di Georuggine.

Capitolo 2

Architettura generale e modello a blocchi

2.1 Scomposizione in crate e confini di responsabilità

Il progetto Georuggine è strutturato come un unico workspace Cargo organizzato secondo una rigida separazione tra logica di dominio agnostica e binari applicativi di sistema.

```
1 PROJECT/
2 |-- Cargo.toml           # Configurazione di compilazione, dipendenze e profili
   release
3 |-- Cargo.lock           # Lockfile deterministico delle dipendenze
4 |-- torino_asti.csv       # File traccia telemetrica GPS reale (tratta urbana
   Torino centro)
5 |-- src/
6 |   |-- lib.rs           # Shared Domain Crate: tipi base, geodesia, protocollo
   wire
7 |   |-- time.rs          # Astrazione temporale e TimeSource per testing
   deterministico
8 |   |-- terminal.rs       # Gestione TTY crudo, RawModeGuard e InputHistory
9 |   |-- server/          # Moduli di libreria del Server
10 |     |-- mod.rs         # Esportazione moduli del server
11 |     |-- state.rs       # Stato globale UserRegistry, FSM, analytics, SQLite
12 |     |-- handler.rs     # Session handler client, tokio::select! loop, framing
   wire
13 |     |-- logger.rs      # Monitoraggio CPU 120s, ring buffer, istogramma ASCII
14 |     |-- cli.rs         # Console shell amministrativa interattiva
15 |     |-- error.rs       # Gerarchia tipizzata degli errori di dominio
   (ServerError)
16 |     |-- runtime.rs     # Server runtime orchestrator e bind TCP
17 |   |-- client/         # Moduli di libreria del Client
18 |     |-- mod.rs         # Esportazione moduli del client
19 |     |-- state.rs       # ClientState e rilevamento transizioni Stopped/Moving
20 |     |-- cli.rs         # Interfaccia CLI conducente e autenticazione
21 |     |-- movement.rs    # PositionProvider e FilePositionProvider da file CSV
22 |     |-- network.rs     # Codec di rete asincrono per linee JSON (ServerReader)
23 |     |-- runtime.rs     # ClientRuntime, loop tokio::select! e gestione eventi
24 |   |-- bin/
25 |     |-- server/main.rs  # Entrypoint eseguibile server (parsing CLI --addr)
26 |     |-- client/main.rs  # Entrypoint eseguibile client (parsing CLI --addr)
27 |-- tests/              # Suite di test di integrazione ed end-to-end
```

```

28      |-- common/mod.rs          # Fixture condivise, helper TestDatabase e coordinate
      campione
29      |-- integration_domain.rs  # Test di integrazione dominio (6 test)
30      |-- integration_client.rs  # Test harness movimento client (6 test)
31      |-- integration_server.rs  # Test harness di rete e persistenza server (20 test)
32      '-- e2e_system_test.rs     # Test harness di sistema end-to-end (6 test)

```

Listing 2.1: Albero dei sorgenti del workspace Georuggine.

2.1.1 Ruoli dei moduli principali

- 1. Libreria di dominio condivisa (georuggine — src/lib.rs):** Non contiene codice di I/O o dipendenze di rete. Definisce le strutture geometriche ([Position](#)), i calcoli geodetici ([haversine_distance_km](#), [euclidean_distance](#)), gli stati FSM ([UserState](#)), le statistiche ([UserStats](#)), le finestre temporali ([TimeWindow](#)) e i messaggi wire ([ClientMessage](#), [ServerMessage](#)). Sia il client che il server compilano questo modulo come dipendenza comune.
- 2. Sottosistema server (src/server/ e src/bin/server/):** Ospita il server multi-tenant asincrono basato su Tokio. Ascolta le connessioni TCP in ingresso sulla porta 0.0.0.0:8080, alloca un green task per ciascuna connessione, gestisce la FSM con soglia di stasi a 180 secondi, calcola le statistiche di viaggio a singolo passaggio, persiste i dati su SQLite, raccoglie le metriche di carico CPU ogni 120 secondi, e fornisce all'amministratore una shell CLI interattiva non bloccante basata su Crossterm ([RawModeGuard](#), [InputHistory](#) in [src/terminal.rs](#)).
- 3. Sottosistema client (src/client/ e src/bin/client/):** Rappresenta l'unità di bordo dell'autoveicolo. Gestisce l'autenticazione iniziale dell'autista, attiva il loop telemetrico periodico ogni 30 secondi utilizzando il provider di movimento ([PositionProvider](#), [FilePositionProvider](#)), ascolta i messaggi broadcast/diretti in arrivo dalla centrale logistica tramite [ServerReader](#) e inoltra comunicazioni testuali digitate dal conducente tramite terminale interattivo non bloccante con cronologia comandi ([InputHistory](#) in [src/terminal.rs](#)).
- 4. Suite di test multi-livello (tests/):** Contiene le fixture condivise in [common/mod.rs](#) (tra cui [TestDatabase](#) con directory isolate [tempfile](#)), i test di integrazione del dominio in [integration_domain.rs](#) e gli harness per test di rete, client ed end-to-end.

2.2 Architettura di contesto C4, modello di comunicazione e deployment

L'architettura del sistema Georuggine è stata concepita per soddisfare simultaneamente i requisiti di affidabilità industriale, reattività in tempo reale e rigida separazione delle responsabilità. La strutturazione a blocchi illustrata nel diagramma **C4 Level 2 (Container Decomposition)** di Figura 2.1 risponde a precise motivazioni ingegneristiche:

- **Disaccoppiamento tra unità di bordo e centrale logistica:** I terminali di bordo (`georuggine-client`) operano in un ambiente distribuito caratterizzato da connettività radiomobile intermittente e risorse limitate. Il client è strutturato come un'entità autonoma che astrae la cinematica del mezzo tramite il pattern Strategy, gestendo localmente l'acquisizione delle coordinate e demandando al server la validazione temporale e l'aggregazione statistica.
- **Modello a green thread e I/O asincrono non bloccante:** Il server centrale (`georuggine-server`) adotta il runtime Tokio con architettura event-driven $M : N$. Ciascuna connessione TCP attiva è gestita da un task cooperativo leggero (≈ 300 byte di overhead), consentendo al server di scalare senza incorrere nei limiti di memoria e context switching tipici dei thread nativi dell'OS.
- **Isolamento della persistenza e protezione dell'event loop:** Le transazioni su memoria di massa (database SQLite e append log CPU) sono rigorosamente segregate nel pool thread `tokio::task::spawn_blocking`. In questo modo, le latenze di I/O sincrone su disco non introducono mai *jitter* nell'elaborazione dei frame telemetrici o nel routing dei messaggi broadcast.
- **Integrità dello stato concorrente via Snapshot Pattern:** Il registro della flotta è mantenuto in RAM tramite `Arc<RwLock<UserRegistry>`. La contesa tra letture frequenti (calcolo metriche, lista veicoli, inoltro chat) e scritture puntuali (aggiornamenti GPS a 30s) viene azzerata rilasciando immediatamente il lock dopo aver clonato le strutture strettamente necessarie.

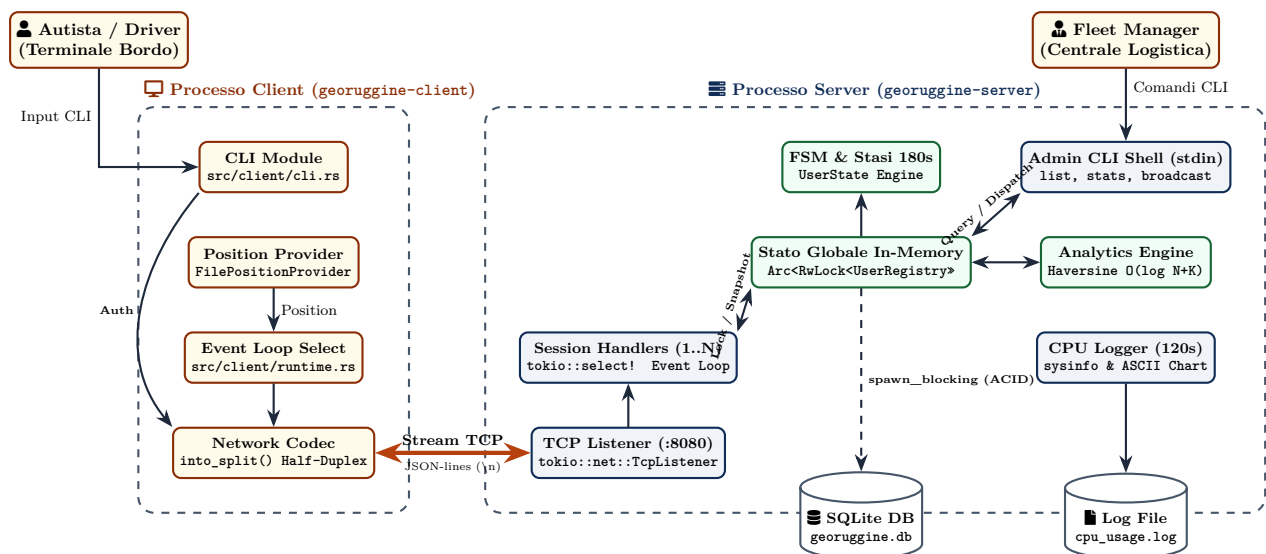


Figura 2.1: Diagramma di contesto e decomposizione funzionale a blocchi (modello C4 Level 2).

Come emerge dalla Figura 2.1, l'interazione tra i componenti si articola secondo percorsi deterministici e privi di contesa:

- Ingresso del dato telemetrico:** Il generatore cinematico del client emette una nuova coordinata ogni 30s; il task telemetrico la incapsula in un messaggio wire inviandola attraverso il codec TCP.
- Ingestione e transizione FSM:** Il `Session Handler` del server riceve il frame JSON-lines, valida la monotonicità temporale del timestamp UTC e aggiorna atomicamente lo stato in memoria. Il motore FSM verifica se lo scostamento spaziale o il tempo di stasi (≥ 180 s) richiedono la commutazione tra `Moving` e `Stopped`.
- Sincronizzazione SQLite:** L'aggiornamento posizionale viene accodato in modo trasparente sul thread pool bloccante, che esegue la query SQL di `INSERT` sul file locale `georuggine.db` preservando le garanzie ACID.
- Supervisione amministrativa:** La console shell dell'amministratore interroga in tempo reale il registro tramite lo Snapshot Pattern, estraendo metriche aggregate geodetiche (`Analytics Engine`) e dispacciando messaggi diretti o broadcast alle mailbox dei client.

2.2.1 Modello di comunicazione e canali di rete

Georuggine adotta un protocollo di livello applicativo basato su **JSON-lines (NDJSON)** instradato su trasporto affidabile **TCP/IP**. La scelta del protocollo stream TCP puro unito al framing delimitato da newline consente di eliminare l'overhead di incapsulamento HTTP/gRPC, riducendo al minimo la banda consumata dai vettori di bordo. La Tabella 2.1 riassume i canali operativi di comunicazione e di I/O del sistema.

Canale	Trasporto / porta			Formato dati	Semantica di instradamento
Stream telemetrico	TCP	Socket	su	JSON UTF-8 terminato da newline \n (0x0A).	Connessione bidirezionale persistente multiplexata (<i>Duplex Stream</i>).
Shell amministrativa	Standard		I/O	Comandi testuali (list, stats, direct, broadcast).	Elaborazione locale asincrona non bloccante rispetto ai socket di rete via Crossterm.
Persistenza ACID	C-API	SQLite	embedded	Query SQL parametriche con binding posizionale.	Esecuzione sincrona confinata nel pool thread dedicato <code>spawn_blocking</code> .
Diagnostica CPU	System Calls	Kernel	via <code>sysinfo</code>	Valori floating-point f32 (% utilizzo CPU del PID).	Campionamento non intrusivo su timer a 120 secondi con scrittura su file log.

Tabella 2.1: Caratteristiche dei canali di comunicazione e I/O del sistema.

2.2.2 Topologia di deployment distribuito

L'architettura logica a container illustrata nel modello C4 si traduce fisicamente nella topologia di deployment distribuito schematizzata in Figura 2.2. Il sistema prevede nodi periferici eterogenei

installati a bordo dei veicoli di flotta, collegati mediante connettività radiomobile cellulare o satellitare verso un'istanza server centrale.

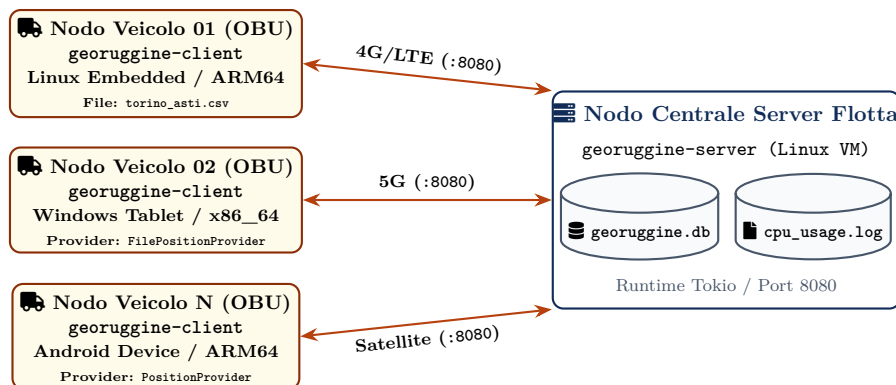


Figura 2.2: Topologia di deployment distribuito con canali radiomobili e nodo centrale.

Grazie all'assenza di dipendenze runtime dinamiche C esterne e alla compilazione statica del motore SQLite, il binario client `georuggine-client` può essere distribuito senza modifiche su architetture eterogenee (sistemi embedded Linux ARM64, tablet Windows x86_64, smartphone Android), garantendo portabilità nativa e facilità di aggiornamento sul campo.

2.3 Schede delle decisioni architetturali fondamentali (ADR-001 — ADR-007)

Le scelte architetturali cardine del sistema Georuggine sono formalizzate secondo il modello standard degli *Architectural Decision Records* (ADR). Ciascun record affronta un problema architetturale critico, esplicita la decisione implementativa adottata in linguaggio Rust e ne sintetizza le conseguenze tecniche e le garanzie formali ottenute. La Tabella 2.2 struttura comparativamente l'intero corpus delle 7 decisioni architetturali, evidenziando l'uniformità dei criteri decisionali e la coerenza ingegneristica dell'architettura complessiva.

Tabella 2.2: Matrice delle decisioni architettureali fondamentali (ADR-001 — ADR-007).

ID & Ambito	Contesto e Problema	Scelta implementativa	Conseguenze e Benefici
ADR-001 Safe Rust (100%)	Requisito di prestazioni native ad alta efficienza computazionale, totale assenza di pause da Garbage Collector (critico per la puntualità dei timer a 30s e 120s) e prevenzione assoluta di corruzione della memoria.	Utilizzo rigoroso ed esclusivo di Rust (Edizione 2021/2024) in modalità 100% Safe Rust. Totale divieto di codice o blocchi unsafe all'interno della logica applicativa.	Eliminazione a tempo di compilazione di intere classi di vulnerabilità (<i>buffer overflow</i> , <i>use-after-free</i> , dangling pointers) e garanzia formale di assenza di data race certificata dal Borrow Checker.
ADR-002 Runtime Tokio (M:N)	Scalabilità multi-tenant per centinaia o migliaia di connessioni concorrenti simultanee; il modello tradizionale 1:1 basato su thread OS alloca 2 ÷ 8 MB di stack per connessione, con insostenibile consumo di RAM e pesante overhead di context switch (1.0 ÷ 2.5 μ s).	Adozione del runtime asincrono Tokio (features = ["full"]) con scheduler multi-threaded <i>work-stealing</i> integrato nativamente con i reactor I/O non bloccanti del sistema operativo (epoll/kqueue/IOCP).	Consumo di memoria ridotto a soli $\approx$ 300 byte per connessione attiva; capacità di gestire oltre 100 000 connessioni simultanee con latenza di dispatch sub-millisecondo.

Continua alla pagina successiva...

Tabella 2.2 (continua dalla pagina precedente)

ID & Ambito	Contesto e Problema	Scelta implementativa	Conseguenze e Benefici
ADR-003 Concorrenza Ibrida	Carico del registro centrale prevalentemente orientato alla lettura (<i>Read-Heavy</i> : query statistiche, inoltro broadcast, ispezione stato flotta) e rare scritture; rischio di stallo qualora un client lento o disconnesso blocchi il ciclo di dispatch.	Stato globale in memoria protetto da <code>Arc<RwLock<UserRegistry></code> con applicazione del rigoroso Snapshot Pattern ; canali MPSC dedicati a capacità limitata a 32 messaggi per client con invio non bloccante via <code>try_send</code> .	Parallelismo di lettura illimitato a contesa zero; immunità da deadlock e backpressure automatica con sollevamento immediato di <code>ServerError::MailboxFull</code> , isolando i client lenti.
ADR-004 SQLite Embedded	Necessità di persistenza transazionale conforme ai requisiti ACID per credenziali utente e tracciamento storico coordinate GPS, senza introdurre la complessità infrastrutturale di un server DBMS esterno di terze parti.	Embedding del driver <code>C sqlite = "0.37"</code> racchiuso nel singolo file portabile <code>georuggine.db</code> . Confinamento di tutte le operazioni di I/O bloccante su filesystem nel thread pool dedicato <code>spawn_blocking</code> .	Database autonomo a zero configurazione; salvaguardia assoluta della reattività dell'event loop asincrono grazie al completo isolamento dell'I/O bloccante.
ADR-005 Wire Protocol JSON-lines	Lo stream continuo TCP (<i>Byte Stream</i>) è privo del concetto intrinseco di confine di messaggio, esponendo la trasmissione a fenomeni di frammentazione o coalescenza dei pacchetti lungo il canale radiomobile.	Meccanismo di framing basato su JSON-lines (ND-JSON) : ogni messaggio è una riga JSON valida serializzata tramite <code>Serde</code> e terminata dal carattere newline ASCII <code>\n</code> (0x0A).	Ingestione asincrona non ambigua mediante <code>BufReader::read_line</code> ; massima facilità di collaudo, diagnostica e ispezione da console tramite strumenti di rete standard (<code>netcat</code>).

Continua alla pagina successiva...

Tabella 2.2 (continua dalla pagina precedente)

ID & Ambito	Contesto e Problema	Scelta implementativa	Conseguenze e Benefici
ADR-006 Footprint Binario	I nodi di bordo periferici operano su hardware embedded con memoria flash e larghezza di banda limitate; imperativo minimizzare il binary footprint per accelerare tempi di boot, deploy e ridurre consumi.	Configurazione del profilo di rilascio in <code>Cargo.toml</code> con direttive di compilazione: <code>opt-level = "z"</code> , <code>fat LTO (lto = true)</code> , <code>codegen-units = 1</code> , <code>panic = "abort"</code> e <code>strip = true</code> .	Contrazione dell'eseguibile server da 37.51 MB a soli 1.35 MB (risparmio netto del 96.4%) senza degradazione delle prestazioni computazionali a runtime.
ADR-007 Terminale Crossterm	L'I/O bloccante standard (<code>stdin().read_line()</code>) blocca interi thread OS, provoca la corruzione visiva della riga di digitazione all'arrivo di messaggi asincroni (<i>Prompt Interleaving</i>) e impedisce la cattura dei tasti freccia Su/Giù per la cronologia.	Integrazione di <code>crossterm</code> con feature <code>event-stream</code> : guard RAII <code>RawModeGuard</code> con ripristino deterministico al drop; multiplexing tastiera in <code>tokio::select!</code> ; sequenze ANSI anti-interleaving (<code>\r\x1B[2K</code>); fallback per ambienti non-TTY.	Esperienza utente interattiva priva di corruzioni grafiche; terminale protetto da disallineamenti; assenza totale di thread bloccati su <code>stdin</code> e gestione uniforme via reactor Tokio.

Capitolo 3

Modello dei dati e invarianti di dominio

3.1 Censimento completo delle strutture dati, enumerazioni e tratti

Il sistema Georuggine adotta un modello di dominio fortemente tipizzato in 100% Safe Rust. Di seguito viene fornito il censimento analitico di tutte le 17 strutture algebriche, enumerazioni, tratti polimorfi e guardie RAII ripartite tra il crate di dominio condiviso (`src/lib.rs`), il sottosistema server (`src/bin/server/`) e il sottosistema client (`src/bin/client/`).

3.1.1 1. Posizione geografica (`Position` — `src/lib.rs`)

Rappresenta un punto geodetico WGS-84 associato a un'impronta temporale UTC. La struttura occupa esattamente 24 byte in memoria stack ($2 \times 8B \text{ f64} + 1 \times 8B \text{ u64}$):

```
1 #[derive(Debug, Clone, Copy, PartialEq, Serialize, Deserialize)]
2 pub struct Position {
3     pub latitude: f64,
4     pub longitude: f64,
5     pub timestamp: u64,
6 }
```

Listing 3.1: Definizione della struct `Position` (`src/lib.rs`).

- **Metodi:** `haversine_distance_km(&self, other: &Position) -> f64` (distanza ortodromica sferica con raggio terrestre $R = 6371.0$ km e clamp anti-NaN), `euclidean_distance(&self, other: &Position) -> f64`.
- **Ruolo architetturale:** Modello geometrico immutabile alla base di telemetria, persistenza SQLite e calcoli analitici.

3.1.2 2. Stato operativo del veicolo (`UserState` — `src/lib.rs`)

Formalizza lo spazio degli stati discreti della macchina a stati finiti (FSM):

```
1 #[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
2 pub enum UserState {
3     Disconnected,
4     Stopped(u64),    // Timestamp Unix di inizio/conferma della sosta
5     Moving(u64),     // Timestamp Unix dell'ultimo spostamento spaziale rilevato
```

```
6 }
```

Listing 3.2: Definizione dell'enum `UserState` (`src/lib.rs`).

- **Costante:** `pub const STOPPED_AFTER_SECS: u64 = 180;` (soglia di stasi a 3 minuti).
- **Ruolo architetturale:** Modello algebrico della dinamica cinematica del veicolo.

3.1.3 3. Finestre temporali di aggregazione (`TimeWindow` — `src/lib.rs`)

Definisce i criteri di partizionamento temporale del calendario UTC:

```
1 #[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
2 pub enum TimeWindow {
3     CurrentDay,
4     CurrentWeek,
5     CurrentMonth,
6 }
```

Listing 3.3: Definizione dell'enum `TimeWindow` (`src/lib.rs`).

- **Metodi:** `lower_bound_timestamp(&self) -> u64` (calcola il confine inferiore deterministico: 00:00:00 UTC del giorno, Lunedì 00:00:00 UTC della settimana, e 1° del mese 00:00:00 UTC via `chrono`).
- **Ruolo architetturale:** Selettore di filtro per le interrogazioni aggregate di flotta.

3.1.4 4. Statistiche aggregate di percorso (`UserStats` — `src/lib.rs`)

Data Transfer Object (DTO) che racchiude le metriche cinematiche calcolate dal motore analitico:

```
1 #[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
2 pub struct UserStats {
3     pub time_window: TimeWindow,
4     pub total_distance_km: f64,
5     pub average_speed_kmh: f64,
6     pub movement_duration_secs: u64,
7     pub pause_duration_secs: u64,
8 }
```

Listing 3.4: Definizione della struct `UserStats` (`src/lib.rs`).

- **Ruolo architetturale:** Risultato delle elaborazioni statistiche richieste dall'amministratore.

3.1.5 5. Messaggi in ingresso al server (`ClientMessage` — `src/lib.rs`)

Payload serializzati inviati dal terminale di bordo verso il server:

```
1 #[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
2 pub enum ClientMessage {
3     Register { username: String, password: String },
4     Login { username: String, password: String },
5     UpdatePosition(Position),
6     SendText { content: String },
7 }
```

Listing 3.5: Definizione dell'enum ClientMessage (src/lib.rs).

3.1.6 6. Messaggi in uscita dal server (ServerMessage — src/lib.rs)

Payload serializzati inviati dal server verso i terminali veicolari connessi:

```
1 #[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
2 pub enum ServerMessage {
3     AuthResult(Result<(), String>),
4     TextMessage { sender: String, content: String },
5     ErrorMessage(String),
6 }
```

Listing 3.6: Definizione dell'enum ServerMessage (src/lib.rs).

3.1.7 7. Gerarchia tipizzata degli errori (ServerError — src/server/error.rs)

Enumerazione a 18 varianti che copre esaustivamente tutti i fallimenti di validazione, autenticazione, persistenza, concorrenza e comunicazione:

```
1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub enum ServerError {
3     InvalidUsernameLength,
4     InvalidUsernameCharacters,
5     PasswordTooShort,
6     PasswordMissingRequiredCharacters,
7     UsernameTaken,
8     InvalidCredentials,
9     AlreadyLoggedIn,
10    Unauthenticated,
11    AlreadyAuthenticated,
12    UserNotFound,
13    OutOfOrderPosition,
14    EmptyMessage,
15    RecipientOffline { recipient: String },
16    MailboxFull { recipient: String },
17    PasswordHashingFailed,
18    PasswordVerificationFailed,
19    PersistenceFailed,
20    BackgroundTaskFailed,
```

```
21 }
```

Listing 3.7: Definizione dell'enum `ServerError` (`src/server/error.rs`).

- **Implementazioni:** `std::fmt::Display` (traduzione in messaggi wire esatti) e `std::error::Error`.
- **Ruolo architetturale:** Separazione netta tra regole di business e codifica dei messaggi di errore.

3.1.8 8. Trait Strategy di movimento (`PositionProvider` — `src/client/movement.rs`)

Interfaccia polimorfa per la generazione/acquisizione delle coordinate di navigazione:

```
1 pub trait PositionProvider {
2     fn next_position(&mut self) -> Option<Position>;
3 }
```

Listing 3.8: Trait `PositionProvider` (`src/client/movement.rs`).

3.1.9 9. Fornitore coordinate da file CSV (`FilePositionProvider` — `src/client/movement.rs`)

Implementazione concreta del pattern Strategy che esegue il replay di una traccia GPS registrata:

```
1 pub struct FilePositionProvider {
2     reader: BufReader<File>,
3     last_position: Option<Position>,
4     time_source: Arc<dyn TimeSource>,
5 }
```

Listing 3.9: Definizione di `FilePositionProvider` (`src/client/movement.rs`).

- **Comportamento:** Legge righe `lat,lon`, associa il timestamp corrente tramite `TimeSource`; all'End-Of-File (EOF) replica l'ultima coordinata con timestamp aggiornato, simulando un veicolo in sosta.

3.1.10 10. Guardia RAI del terminale (`RawModeGuard` — `src/terminal.rs`)

Struttura RAI che garantisce il ripristino deterministico della modalità terminale cooked:

```
1 pub(crate) struct RawModeGuard;
2
3 impl RawModeGuard {
4     pub(crate) fn try_new() -> std::io::Result<Self> {
5         crossterm::terminal::enable_raw_mode()?;
6         Ok(Self)
7     }
8 }
9
10 impl Drop for RawModeGuard {
```

```

11     fn drop(&mut self) {
12         let _ = crossterm::terminal::disable_raw_mode();
13     }
14 }

```

Listing 3.10: Guardia RAI RawModeGuard (src/terminal.rs).

3.1.11 11. Cronologia comandi e navigazione prompt (InputHistory — src/terminal.rs)

Buffer storico per la navigazione dei comandi e messaggi digitati tramite i tasti freccia Su/Giù:

```

1  #[derive(Default, Debug, PartialEq, Eq)]
2  pub(crate) struct InputHistory {
3      pub(crate) entries: Vec<String>,
4      pub(crate) cursor: Option<usize>,
5  }

```

Listing 3.11: Struttura InputHistory (src/terminal.rs).

- **Metodi:** `push(input)` (ignora stringhe vuote e duplicati consecutivi trimmando gli spazi), `up()` -> `Option<&str>` (risale verso il comando precedente con saturazione a 0), `down()` -> `Option<&str>` (scende verso il più recente, deselezionando il cursore oltre l'ultimo).

3.1.12 12. Automa degli eventi da tastiera (KeyAction / ClientKeyAction)

Enumerazione algebrica che modella le transizioni pure generate dalla pressione dei singoli tasti:

```

1  // Server: KeyAction (src/bin/server/cli.rs)
2  #[derive(Debug, PartialEq, Eq)]
3  pub(crate) enum KeyAction {
4      InsertChar(char),
5      Backspace,
6      Redraw,
7      Submit(String),
8      Exit,
9      None,
10 }
11
12 // Client: ClientKeyAction (src/bin/client/cli.rs)
13 #[derive(Debug, PartialEq, Eq)]
14 pub(crate) enum ClientKeyAction {
15     InsertChar(char),
16     Backspace,
17     Redraw,
18     SendMessage(String),
19     Exit,
20     None,
21 }

```

Listing 3.12: Enum KeyAction e ClientKeyAction (cli.rs).

- **Funzioni di transizione:** `process_key_event` e `process_client_key_event` (funzioni pure testabili unitariamente senza I/O).

3.1.13 13. Sottosistema diagnostico CPU (CpuTracker — src/bin/server/logger.rs)

Gestore del monitoraggio periodico delle risorse computazionali del processo server:

```
1 pub struct CpuTracker {
2     pub history: VecDeque<(u64, f32)>,
3     pub max_samples: usize,
4     pub sys: sysinfo::System,
5     pub pid: sysinfo::Pid,
6     pub log_path: PathBuf,
7 }
```

Listing 3.13: Definizione di CpuTracker (src/bin/server/logger.rs).

- **Flag statico:** `pub static AUTO_SHOW_CHART: AtomicBool = AtomicBool::new(false);`
- **Metodi:** `measure_and_push(&mut self) -> f32, render_ascii_chart(&self), log_executable_size()`

3.1.14 14. Stato globale concorrente (ServerState — src/bin/server/state.rs)

Singleton centrale del server che coordina registri in memoria, canali e persistenza su disco:

```
1 #[derive(Clone)]
2 pub struct ServerState {
3     pub registry: Arc<tokio::sync::RwLock<UserRegistry>>,
4     db_path: PathBuf,
5     pub driver_message_sender: tokio::sync::broadcast::Sender<(String, String)>,
6 }
```

Listing 3.14: Definizione di ServerState (src/bin/server/state.rs).

- **Metodi principali:** `register_user, user_login, update_user_position, calculate_user_stats, send_admin_direct, send_admin_broadcast, process_message, remove_active_client.`

3.1.15 15. Registro utenti e sessioni attive (UserRegistry — src/bin/server/state.rs)

Contenitore protetto dal lock in lettura/scrittura asincrono:

```
1 pub struct UserRegistry {
2     pub map: HashMap<String, UserProfile>,
3     pub active_clients: HashMap<String, ClientSender>,
4 }
```

Listing 3.15: Definizione di UserRegistry (src/bin/server/state.rs).

- **Alias di tipo:** `pub type ClientSender = tokio::sync::mpsc::Sender<ServerMessage>;`
(capacità 32).

3.1.16 16. Profilo anagrafico e telemetrico veicolo (UserProfile — src/bin/server/state.rs)

Rappresentazione in-memory del profilo conducente:

```
1 #[derive(Debug, Clone)]
2 pub struct UserProfile {
3     pub username: String,
4     pub password: String,           // Hash crittografico Bcrypt ($2b$12$...)
5     pub state: UserState,           // Stato FSM (Disconnected, Stopped, Moving)
6     pub positions: Vec<Position>,   // Storico posizioni ordinate per timestamp
7 }
```

Listing 3.16: Definizione di UserProfile (src/bin/server/state.rs).

3.1.17 17. Decoder di trame wire client (ServerReader — src/bin/client/network.rs)

Lettore asincrono bufferizzato per lo stream TCP in ricezione sul client:

```
1 pub struct ServerReader {
2     lines: Lines<BufReader<OwnedReadHalf>>,
3 }
4
5 impl ServerReader {
6     pub fn new(read_half: OwnedReadHalf) -> Self { ... }
7     pub async fn read_message(&mut self) -> io::Result<Option<ServerMessage>> { ... }
8 }
```

Listing 3.17: Definizione di ServerReader (src/bin/client/network.rs).

3.2 Serializzazione e deserializzazione Serde

La codifica sul canale wire avviene tramite il framework **Serde** (`serde_json`), che traduce le enumerazioni algebriche Rust con campi nominati nel formato JSON standard.

3.2.1 Esempi di serializzazione wire (JSON-lines)

```
1 // 1. Registrazione account
2 {"Register":{"username":"driver1","password":"SecretPassword1"}}
3
4 // 2. Invio telemetria
5 {"UpdatePosition":{"latitude":45.0703,"longitude":7.6869,"timestamp":1725000000}}
6
7 // 3. Esito autenticazione con successo
8 {"AuthResult":{"Ok":null}}
```

```
9
10 // 4. Esito autenticazione con errore
11 {"AuthResult":{"Err":"Incorrect username or password"}}
12
13 // 5. Messaggio broadcast / diretto centrale
14 {"TextMessage":{"sender":"FleetAdmin","content":"Attenzione: Rallentamenti sulla A4"}}
```

Listing 3.18: Esempi di trame serializzate in formato JSON-lines.

3.3 Invarianti di dominio e regole di validazione

Il sistema garantisce a livello di tipo e mediante asserzioni esplicite le seguenti quattro invarianti di integrità:

1. **Invariante geografica:** Le coordinate devono appartenere al dominio fisico standard WGS-84:

$$\phi \in [-90.0, +90.0]^\circ, \quad \lambda \in [-180.0, +180.0]^\circ$$

2. **Invariante di monotonicità temporale:** Il vettore dello storico posizioni $\mathcal{P} = [P_0, P_1, \dots, P_N]$ è ordinato in senso strettamente non decrescente rispetto al tempo:

$$\forall i \in \{1, \dots, N\}, \quad P_i.\text{timestamp} \geq P_{i-1}.\text{timestamp}$$

3. **Invariante delle finestre di calendario:** Il timestamp di confine inferiore calcolato per qualsiasi finestra soddisfa sempre la disuguaglianza $t_{\text{bound}} \leq t_{\text{now}}$.
4. **Invariante di sicurezza delle credenziali:** La stringa persistita nella colonna `password` di SQLite è sempre una stringa Bcrypt valida a 60 caratteri con prefisso `$2b$12$` o `$2a$12$`, generata con sale crittografico a 128 bit.

3.4 Verifica empirica e matrice dei test del modello dati

La correttezza delle strutture dati, dei contratti di serializzazione Serde, delle funzioni di validazione e delle invarianti di sicurezza delle credenziali è comprovata dai test automatizzati censiti in Tabella 3.1.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e invarianti validate
1	test_client_message_json_contract	tests/ integration_ domain.rs	Contratto JSON wire: serializzazione e deserializzazione conforme di ClientMessage .
2	test_server_message_json_contract	tests/ integration_ domain.rs	Contratto JSON wire: verifica schemi di tutti i rami di ServerMessage .
3	malformed_protocol_messages_are_rejected	tests/ integration_ domain.rs	Robustezza parser: scarto di frame con JSON malformato o chiavi mancanti.
4	accepts_credential_boundary_values	src/bin/server/ state.rs	Limiti esatti username (4 e 20 caratteri) e password (≥ 8 caratteri).
5	login_rejects_invalid_credentials	src/bin/server/ state.rs	Matrice di rigetto su combinazioni illegali (spazi, caratteri non conformi).
6	hash_password_round_trip	src/bin/server/ state.rs	Cifratura Bcrypt: hash a 60 caratteri con salt a 128 bit e verifica positiva della chiave.
7	registration_persists_hashed_user	src/bin/server/ state.rs	Invariante di sicurezza: la password nel DB SQLite è memorizzata solo come hash Bcrypt.
8	registration_rejects_invalid_credentials	src/bin/server/ state.rs	Protezione DB: rigetto credenziali non conformi prima di qualsiasi operazione SQL.
9	registration_trims_username_before_persisting	src/bin/server/ state.rs	Normalizzazione dati: rimozione di spazi bianchi iniziali e finali prima della persistenza.
10	rejects_duplicate_registration	src/bin/server/ state.rs	Invariante di unicità: rigetto registrazione con username già persistito su SQLite.
11	accepted_credentials_can_log_in	src/bin/server/ state.rs	Coerenza end-to-end: credenziali salvate consentono il successivo login con successo.
12	login_rejects_wrong_credentials	src/bin/server/ state.rs	Rigetto password errata con preservazione dello stato in memoria e su DB.
13	parse_blanks / parse_trims / parse_unicode	src/bin/client/ cli.rs	Sanitizzazione stringhe: gestione input vuoti, spazi multipli e caratteri UTF-8 estesi.

Tabella 3.1: Matrice dei test per il Modello Dati e le Invarianti di Dominio.

Capitolo 4

Protocollo di rete wire e comunicazione

4.1 Meccanismo di framing JSON-lines su stream TCP

Nei protocolli operanti su socket TCP stream, non esiste alcuna corrispondenza 1:1 garantita tra una singola invocazione di `write()` da parte del mittente e una singola `read()` del destinatario. Fenomeni quali la segmentazione IP, l'algoritmo di Nagle e le dimensioni delle finestre TCP possono unire più messaggi in un unico blocco (*packet coalescing*) o spezzare un singolo messaggio su letture distinte (*packet fragmentation*).

Georuggine risolve il framing in modo robusto e minimale adottando la convenzione **JSON-lines (NDJSON)**:

1. Ciascun messaggio è un oggetto JSON serializzato privo di ritorni a capo interni non escapati.
2. Ciascun frame è delimitato in chiusura dal carattere ASCII newline `\n` (0x0A).
3. L'eventuale carattere carriage return `\r` (0x0D) precedente viene automaticamente ignorato dal parser.

4.2 Ingestione asincrona bufferizzata e multiplexing

Nel modulo `handler.rs`, la connessione TCP `tokio::net::TcpStream` viene suddivisa nelle due metà indipendenti di lettura e scrittura tramite `tokio::io::split(socket)`:

- La metà di lettura viene incapsulata in un `tokio::io::BufReader<ReadHalf>`, che gestisce internamente un buffer circolare di ricezione.
- La funzione `reader.read_line(&mut line)` legge ed accumula i byte in arrivo fino a intercettare il delimitatore `\n`, garantendo che alla routine di deserializzazione giunga sempre una riga completa.

L'event loop della sessione client è strutturato attorno alla macro `tokio::select!`, che realizza un multiplexing concorrente a due rami:

1. **Ramo inbound (rete → server)**: Monitora l'arrivo di nuovi dati dal socket del client via `reader.read_line()`.
2. **Ramo outbound (centrale → client)**: Monitora la ricezione di messaggi diretti o broadcast dalla mailbox MPSC associata alla sessione via `client_rx.recv()`.

```
1 loop {
2     line.clear();
3     tokio::select! {
4         // Ramo 1: Ingestione pacchetti dal socket TCP del veicolo
5         bytes_res = reader.read_line(&mut line) => {
6             let bytes = bytes_res?;
7             if bytes == 0 {
8                 return Ok(()); // EOF: Connessione chiusa dal client (teardown
// deterministico)
9             }
10            let client_msg: ClientMessage = match serde_json::from_str(&line) {
11                Ok(msg) => msg,
12                Err(err) => {
13                    let err_msg = ServerMessage::ErrorMessage(
14                        format!("Invalid JSON payload format: {}", err)
15                    );
16                    send_server_message(&mut write_half, &err_msg).await?;
17                    continue;
18                }
19            };
20            // Smistamento ed elaborazione tipizzata del comando...
21        }
22
23        // Ramo 2: Smistamento messaggi in uscita dalla mailbox verso il client
24        Some(msg) = async {
25            match client_rx.as_mut() {
26                Some(rx) => rx.recv().await,
27                None => std::future::pending().await,
28            }
29        } => {
30            send_server_message(&mut write_half, &msg).await?;
31        }
32    }
33 }
```

Listing 4.1: Loop di multiplexing asincrono (src/server/handler.rs).

4.2.1 Semantiche del protocollo: credenziali, broadcast e shutdown

- **Asimmetria di validazione delle credenziali:** Il client (src/client/cli.rs) effettua una pre-validazione unicamente sulla presenza (stringhe di username e password non vuote); la validazione stringente dei vincoli di lunghezza (4–20 caratteri alfanumerici) e di complessità della password (≥ 8 caratteri con almeno una minuscola, una maiuscola e una cifra) viene eseguita in modo autoritativo ed esclusivo dal server centrale in `validate_credentials` (src/server/state.rs).

- ▶ **Erogazione broadcast non bloccante (best-effort):** La trasmissione di messaggi broadcast a tutti i veicoli attivi avviene tramite iterazione non bloccante sui canali MPSC limitati (`try_send`, capacità 32 messaggi). Qualora la coda di ricezione di un terminale lento sia satura (`TrySendError::Full`), il messaggio per quel veicolo viene scartato per prevenire il rallentamento del server (difesa anti-Slowloris), incrementando il contatore di tentativi completati.
- ▶ **Semantica di shutdown e chiusura socket:** Il protocollo wire non prevede una variante esplicita di messaggio `Shutdown` in `ServerMessage`. La chiusura della sessione è affidata al teardown deterministico del canale TCP: alla ricezione di EOF (`bytes == 0`) o alla caduta fisica della connessione, il runtime dealloca la sessione e ripristina lo stato in `UserState::Disconnected`.

4.3 Diagrammi di sequenza del protocollo di comunicazione

I seguenti diagrammi di sequenza vettoriali TikZ descrivono i sei scenari operativi cardine del protocollo di rete di Georuggine:

4.3.1 1. Handshake TCP e registrazione nuovo utente

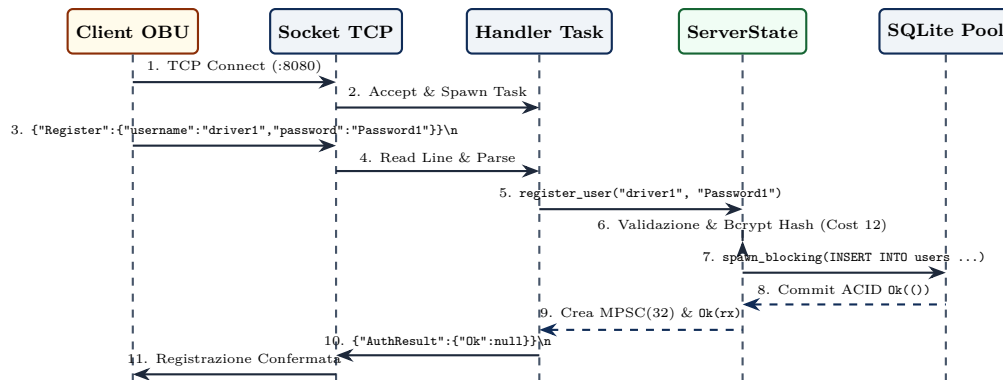


Figura 4.1: Diagramma di sequenza dell'handshake TCP e registrazione nuovo account.

4.3.2 2. Autenticazione (Login) con verifica Bcrypt e blocco accessi duplicati

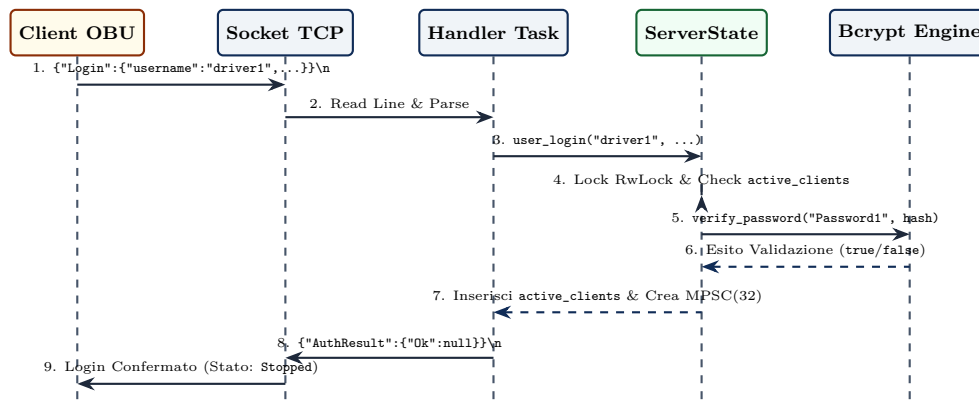


Figura 4.2: Diagramma di sequenza di autenticazione e mutua esclusione delle sessioni attive.

4.3.3 3. Trasmissione periodica telemetria (30s) e aggiornamento FSM

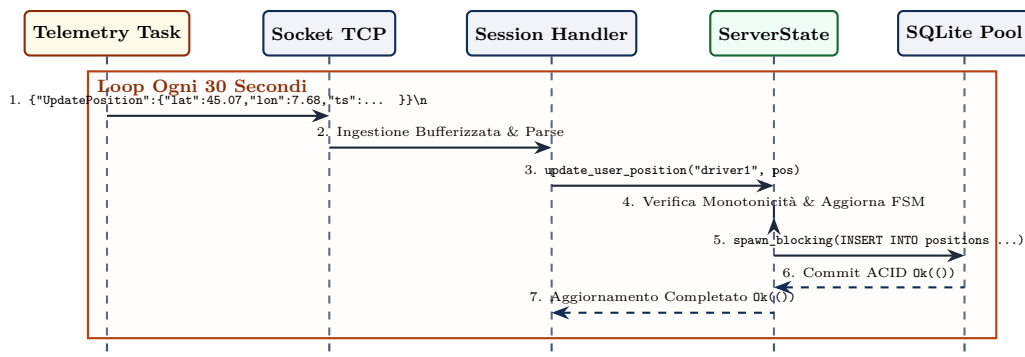


Figura 4.3: Diagramma di sequenza del ciclo periodico di invio telemetrico (30s).

4.3.4 4. Messaggistica broadcast e diretta (unicast) dalla centrale

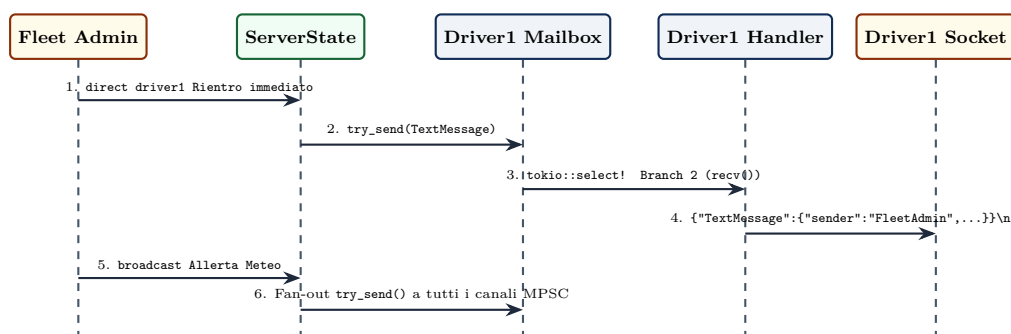


Figura 4.4: Diagramma di sequenza del recapito messaggi diretti (*unicast*) e broadcast.

4.3.5 5. Ricezione messaggi da terminale autista con multiplexing asincrono

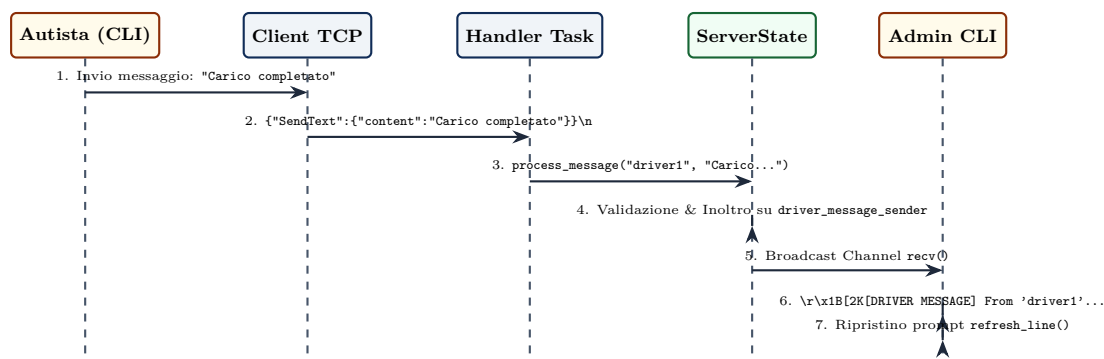


Figura 4.5: Diagramma di sequenza di ricezione e multiplexing messaggi autista verso centrale.

4.3.6 6. Disconnessione pulita (EOF) e gestione timeout

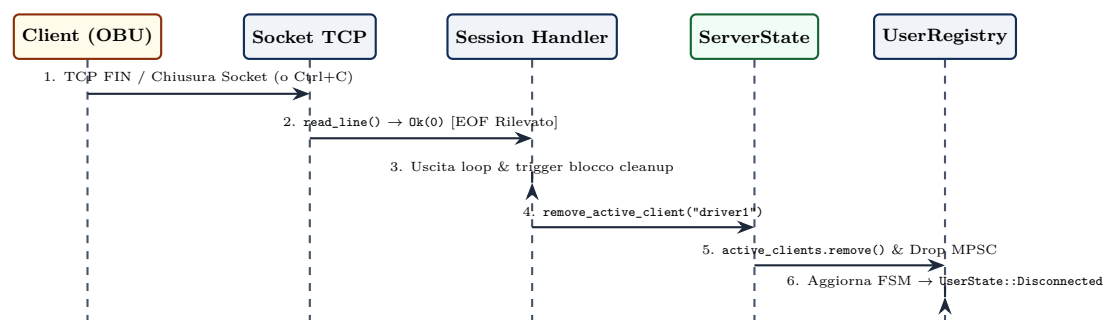


Figura 4.6: Diagramma di sequenza della chiusura pulita su EOF e deallocazione risorse.

4.4 Matrice completa di gestione degli errori (ServerError) e del trasporto

La Tabella 4.1 documenta in maniera esaustiva la mappatura di tutte le 18 varianti dell'enum `ServerError` e delle condizioni di errore di trasporto e framing sul protocollo wire e sulle azioni di gestione della sessione.

Tabella 4.1: Matrice completa di gestione degli errori di dominio (`ServerError`) e di protocollo wire.

#	Variante <code>ServerError</code> / Condizione	Causa scatenante	Payload wire / Output inviato	Azione sessione / socket
1	<code>InvalidUsernameLength</code>	Lunghezza username < 4 o > 20 caratteri	<code>AuthResult(Err("Validation error: Username must be between 4 and 20..."))</code>	Registrazione respinta; socket aperto per retry.
2	<code>InvalidUsernameCharacters</code>	Username contiene caratteri non alfanumerici	<code>AuthResult(Err("Validation error: Username must be alphanumeric"))</code>	Registrazione respinta; socket aperto.
3	<code>PasswordTooShort</code>	Password < 8 caratteri	<code>AuthResult(Err("Validation error: Password must be at least 8..."))</code>	Registrazione respinta; socket aperto.
4	<code>PasswordMissingRequiredCharacters</code>	Password priva di min, maiusc o cifra	<code>AuthResult(Err("Validation error: Password must contain..."))</code>	Registrazione respinta; socket aperto.
5	<code>UsernameTaken</code>	Account già presente nel database SQLite	<code>AuthResult(Err("Registration failed: username is already registered"))</code>	Registrazione respinta; socket aperto.
6	<code>InvalidCredentials</code>	Password errata o account inesistente	<code>AuthResult(Err("Incorrect username or password"))</code>	Login fallito; socket aperto per nuovo tentativo.
7	<code>AlreadyLoggedIn</code>	Tentativo login su utente già in <code>active_clients</code>	<code>AuthResult(Err("User is already logged in from another session"))</code>	Secondo login respinto; prima sessione preservata.
8	<code>Unauthenticated</code>	Invio comandi protetti prima del login	<code>ErrorMessage("Access denied: Client is not authenticated...")</code>	Comando respinto; attesa autenticazione.
9	<code>AlreadyAuthenticated</code>	Secondo login inviato sullo stesso socket	<code>ErrorMessage("You are already logged in on this session.")</code>	Comando ignorato; sessione mantenuta.

Continua alla pagina successiva...

Tabella 4.1 (continua dalla pagina precedente)

#	Variante ServerError / Condizione	Causa scatenante	Payload wire / Output inviato	Azione sessione / socket
10	UserNotFound	Query statistiche su driver inesistente	Errore CLI / <code>ErrorMessage("User profile not found")</code>	Notifica amministratore; nessun crash.
11	OutOfOrderPosition	Timestamp GPS $t_{\text{new}} < t_{\text{last}}$	<code>ErrorMessage("Invalid timestamp: Point is older than last...")</code>	Punto scartato; FSM e DB invariati.
12	EmptyMessage	Messaggio chat vuoto o soli spazi	<code>ErrorMessage("Validation error: Message content cannot be empty...")</code>	Messaggio scartato; nessun broadcast.
13	RecipientOffline	Destinatario unicast non connesso	Notifica CLI: Driver <code>'{recipient}' is not currently connected.</code>	Messaggio scartato; server attivo.
14	MailboxFull	Coda canale MPSC client (32 msg) satura	Gestione non bloccante via <code>try_send</code>	Messaggio scartato; nessun blocco thread.
15	PasswordHashingFailed	Errore interno algoritmo Bcrypt	<code>AuthResult(Err("Server error: failed to hash password"))</code>	Errore interno; socket mantenuto.
16	PasswordVerification Failed	Stringa hash Bcrypt corrotta	<code>AuthResult(Err("Server error: failed to verify password"))</code>	Errore interno; socket mantenuto.
17	PersistenceFailed	Errore SQLite I/O o tabella occupata	<code>AuthResult(Err(...)) / ErrorMessage(...)</code>	Errore loggato; isolamento <code>spawn_blocking</code> .
18	BackgroundTaskFailed	Panic o join error nel task bloccante	<code>AuthResult(Err(...)) / ErrorMessage(...)</code>	Runtime Tokio protetto da panico task.
19	Framing / JSON malformato	Errore di sintassi riga JSON	<code>ErrorMessage("Invalid JSON payload format: ...")</code>	Frame scartato; buffer ripulito; socket aperto.
20	Chiusura TCP / EOF	Rilevamento <code>read_line == 0</code>	Nessuna (rilevamento EOF a livello socket)	Deallocazione MPSC, cleanup sessione, FSM → Disconnected .

4.5 Verifica empirica e matrice dei test del protocollo di rete

La robustezza del trasporto TCP a livello di framing, la gestione deterministica degli errori wire e l'isolamento dei canali di comunicazione sono convalidati dai test di unità e di integrazione censiti in Tabella 4.2.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e comportamento convalidato
1	reader_accepts_crlf	src/client/ network.rs	Framing multi-piattaforma: accettazione trasparente sia di terminatori CRLF (\r\n) che LF (\n).
2	reader_decodes_fragmented_messages_in_order	src/client/ network.rs	Segmentazione TCP: ricomposizione e decodifica ordinata di frame inviati a frammenti sparsi.
3	reader_recovers_after_bad_message	src/client/ network.rs	Resilienza socket: scarto di riga corrotta e recupero immediato sul messaggio valido successivo.
4	reader_returns_none_at_eof	src/client/ network.rs	Chiusura socket: rilevamento deterministico di EOF con ritorno di None senza panico.
5	send_fails_when_peer_is_closed	src/client/ network.rs	Gestione broken pipe: fallimento controllato con errore I/O se il socket remoto è chiuso.
6	send_message_writes_one_json_line	src/client/ network.rs	Invariante wire: ogni messaggio serializzato occupa esattamente una riga terminata da \n.
7	recovers_after_bad_json	src/server/ handler.rs	Trattamento payload errato: invio di ErrorMessage preservando attiva la sessione TCP.
8	rejects_bad_login	src/server/ handler.rs	Autenticazione: credenziali errate restituiscono <code>AuthResult(Err)</code> lasciando il socket aperto per retry.
9	rejects_unauthenticated_position	src/server/ handler.rs	Controllo accessi: telemetria ricevuta prima del login respinta con errore di protocollo.
10	rejects_unauthenticated_text	src/server/ handler.rs	Controllo accessi: messaggi di testo inviati prima dell'autenticazione scartati con errore.
11	cleans_up_on_eof	src/server/ handler.rs	Rilascio risorse: deallocazione automatica di mailbox MPSC e rimozione da <code>active_clients</code> su EOF.
12	tcp_framing_roundtrip	tests/ integration_ client.rs	Integrazione client TCP: serializzazione, invio, lettura e deserializzazione su canale reale duplex.
13	tcp_eof_is_clean	tests/ integration_ client.rs	Bonifica socket: chiusura lato client gestita con deallocazione completa senza memory leak.
14	tcp_authentication_lifecycle	tests/ integration_ server.rs	Flusso completo: connessione socket, handshake di registrazione, disconnessione e login successivo.
15	tcp_recovers_after_malformed_input	tests/ integration_ server.rs	Test di robustezza: iniezione di byte casuali e verifica che i messaggi successivi siano processati.

Tabella 4.2: Matrice dei test per il Protocollo di Rete Wire, Framing e Trasporto.

Capitolo 5

Macchina a stati finiti (FSM) del veicolo e dinamica temporale

5.1 Formalizzazione matematica dell'automa a stati

La dinamica operativa di ciascun veicolo della flotta è formalizzata come una Macchina a Stati Finiti Deterministica continua nel tempo:

$$\mathcal{M} = \langle S, \Sigma, \delta, s_0, F \rangle \quad (5.1)$$

► **Spazio degli stati (S):**

$$S = \{\text{Disconnected}, \text{Stopped}(t_{\text{stop}}), \text{Moving}(t_{\text{last_moved}})\}$$

- **Disconnected:** Il veicolo non possiede una sessione di rete attiva sul server.
- **Stopped(t_{stop}):** Il veicolo è connesso e fermo; $t_{\text{stop}} \in \mathbb{N}_0$ è il timestamp Unix in cui è stato rilevato l'arresto.
- **Moving($t_{\text{last_moved}}$):** Il veicolo è connesso e in marcia; $t_{\text{last_moved}} \in \mathbb{N}_0$ è il timestamp Unix dell'ultimo spostamento cinematico rilevato.

► **Alfabeto degli eventi (Σ):**

$$\Sigma = \{\text{EvConnect}(t), \text{EvDisconnect}, \text{EvUpdatePosition}(P_t)\}$$

dove $P_t = (\phi_t, \lambda_t, t)$ è la coordinata GPS ricevuta al timestamp t .

► **Stato iniziale:** $s_0 = \text{Disconnected}$.

► **Insieme degli stati finali:** $F = \emptyset$ (automa reattivo a ciclo continuo).

5.2 Logica di transizione e condizioni di guardia

Sia $P_t = (\phi_{\text{new}}, \lambda_{\text{new}}, t_{\text{new}})$ la nuova posizione ricevuta e $P_{\text{last}} = (\phi_{\text{last}}, \lambda_{\text{last}}, t_{\text{last}})$ l'ultima posizione registrata nel profilo utente. Definiamo il predicato booleano di spostamento spaziale $\text{Moved}(P_t, P_{\text{last}})$ come:

$$\text{Moved}(P_t, P_{\text{last}}) \iff (\phi_{\text{new}} \neq \phi_{\text{last}}) \vee (\lambda_{\text{new}} \neq \lambda_{\text{last}}) \quad (5.2)$$

La funzione di transizione globale $\delta : S \times \Sigma \rightarrow S$ è formalmente definita come segue:

1. **Eventi di connessione e disconnessione:**

$$\delta(\text{Disconnected}, \text{EvConnect}(t)) = \text{Stopped}(t)$$

$$\delta(s, \text{EvDisconnect}) = \text{Disconnected} \quad \forall s \in S$$

2. Primo pacchetto telemetrico (storico vuoto):

$$\delta(s, \text{EvUpdatePosition}(P_t)) = \text{Stopped}(t_{\text{new}}) \quad \text{se } \text{positions} = \emptyset$$

3. Transizioni dallo stato **Stopped**(t_0):

$$\delta(\text{Stopped}(t_0), \text{EvUpdatePosition}(P_t)) = \begin{cases} \text{Moving}(t_{\text{new}}), & \text{se } \text{Moved}(P_t, P_{\text{last}}) \\ \text{Stopped}(t_0), & \text{se } \neg \text{Moved}(P_t, P_{\text{last}}) \end{cases}$$

4. Transizioni dallo stato **Moving**(t_m) (regola di stasi a 180 secondi):

$$\delta(\text{Moving}(t_m), \text{EvUpdatePosition}(P_t)) = \begin{cases} \text{Moving}(t_{\text{new}}), & \text{se } \text{Moved}(P_t, P_{\text{last}}) \\ \text{Stopped}(t_{\text{new}}), & \text{se } \neg \text{Moved}(P_t, P_{\text{last}}) \wedge (t_{\text{new}} - t_m \geq 180) \\ \text{Moving}(t_m), & \text{se } \neg \text{Moved}(P_t, P_{\text{last}}) \wedge (t_{\text{new}} - t_m < 180) \end{cases}$$

5.3 Dinamica temporale: soglia di stasi (180s) e cadenza telemetrica (30s)

L'interazione tra l'intervallo di campionamento telemetrico ($T_{\text{sample}} = 30 \text{ s}$) e la soglia di stasi della FSM ($T_{\text{stasis}} = 180 \text{ s}$) garantisce che le fermate temporanee tipiche del traffico urbano o dei semafori non vengano erroneamente conteggiate come soste di fine viaggio:

- **Inizializzazione al primo campionamento** ($t = 0 \text{ s}$): All'avvio della sessione e ricezione della prima coordinata P_0 , non sussistendo una posizione precedente con cui comparare lo spostamento spaziale, la FSM registra il veicolo nello stato **Stopped**(t_0). La prima transizione a **Moving**(t_1) ha luogo al campionamento successivo ($t = 30 \text{ s}$) qualora $P_1 \neq P_0$.
- **Soste brevi** ($< 180 \text{ s}$, es. **semaforo rosso da 60s o 120s**): Il veicolo in moto si arresta al semaforo. Ai campionamenti $t_m + 30\text{s}$, $t_m + 60\text{s}$, $t_m + 90\text{s}$, $t_m + 120\text{s}$ e $t_m + 150\text{s}$, le coordinate rimangono invariate. La condizione di guardia ($t_{\text{new}} - t_m < 180$) mantiene il veicolo nello stato **Moving**(t_m).
- **Soste prolungate** ($\geq 180 \text{ s}$, es. **parking o carico merci**): Al raggiungimento di $t_m + 180\text{s}$ di coordinate invariate (pari a 6 campionamenti consecutivi stazionari), la guardia ($t_{\text{new}} - t_m \geq 180$) scatta: il server riconosce l'arresto prolungato e commuta lo stato a **Stopped**($t_m + 180$).
- **Ripartenza immediata**: Alla successiva variazione di coordinate al timestamp t_{restart} , la FSM transita istantaneamente a **Moving**(t_{restart}).

5.4 Diagramma di stato e timeline temporale

5.4.1 Diagramma degli stati FSM

La Figura 5.1 mostra l'automa a stati finiti vettoriale con tutte le condizioni di guardia e gli eventi di transizione.

5.4.2 Diagramma temporale (timeline) della FSM

La Figura 5.2 illustra l'evoluzione dello stato cinematico e del timer di stasi durante un ciclo di campionamento reale di 210 secondi.

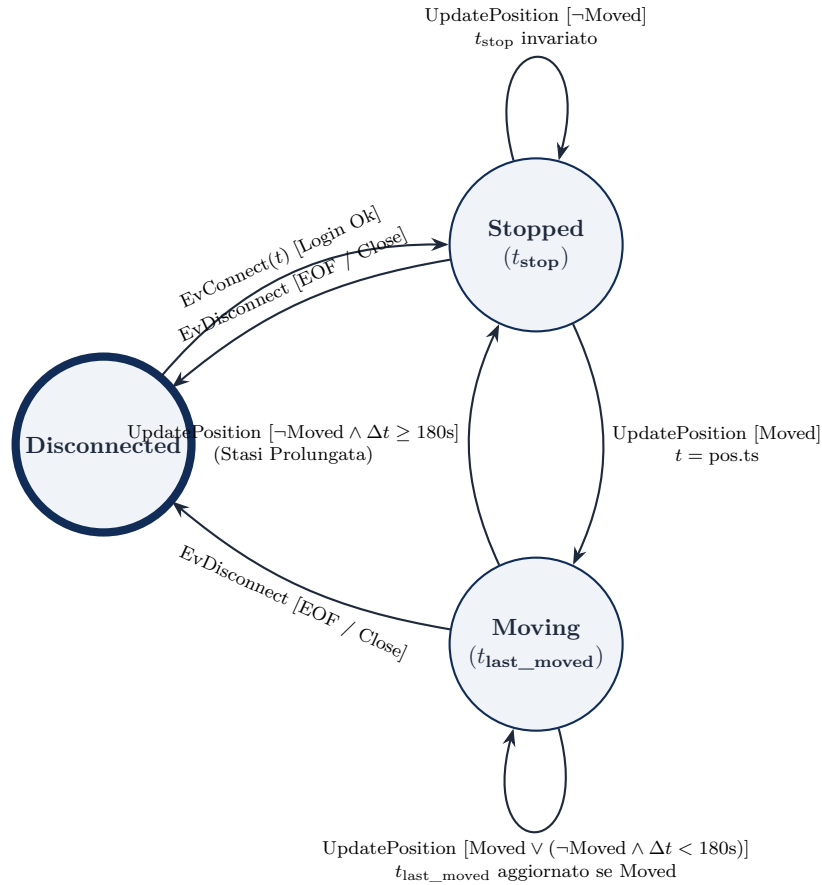


Figura 5.1: Macchina a stati finiti (FSM) del monitoraggio cinematico del veicolo.

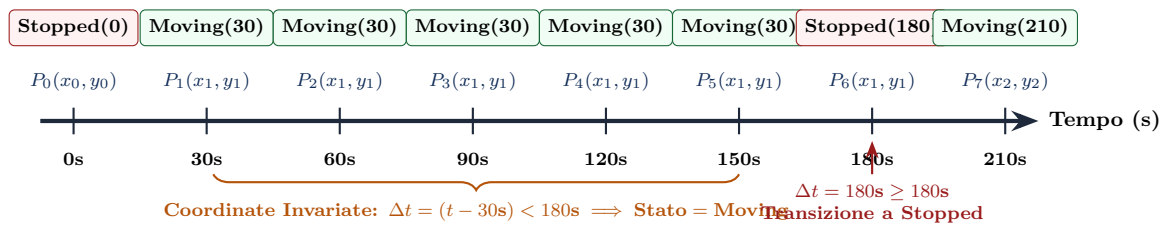


Figura 5.2: Timeline temporale della FSM con campionamento a 30s e commutazione alla soglia di stasi (180s).

5.5 Architettura del terminale CLI e gestione interattiva dell'input (Crossterm)

I moduli `src/bin/client/cli.rs` e `src/bin/server/cli.rs` condividono una struttura architetturale simmetrica fondata sul disaccoppiamento tra manipolazione dello stato di input ed effetti di I/O, risolvendo alla radice il problema del *Terminal Prompt Interleaving*.

5.5.1 Il pattern RAII `RawModeGuard`

L'accesso alla modalità raw richiede la certezza assoluta di rilascio per evitare che il terminale dell'utente rimanga in uno stato corrotto (assenza di echo e newline spezzati) in caso di errore, panic o terminazione forzata (`Ctrl+C`). In Rust, ciò si realizza tramite l'invariante RAII (*Resource Acquisition Is Initialization*):

```
1  /// RAII guard ensuring raw mode is automatically disabled on drop/exit.
2  pub(crate) struct RawModeGuard;
3
4  impl RawModeGuard {
5      pub(crate) fn try_new() -> std::io::Result<Self> {
6          crossterm::terminal::enable_raw_mode()?;
7          Ok(Self)
8      }
9  }
10
11 impl Drop for RawModeGuard {
12     fn drop(&mut self) {
13         let _ = crossterm::terminal::disable_raw_mode();
14     }
15 }
```

Listing 5.1: Guardia RAII per la gestione atomica della Raw Mode (`src/bin/client/cli.rs`).

La struct `RawModeGuard` è una *Zero-Sized Type* (ZST) a costo zero di memoria. L'invocazione di `try_new()` attiva la modalità raw via `crossterm`, mentre il compilatore inserisce deterministicamente la chiamata al metodo `drop()` su ogni punto di uscita dallo scope.

5.5.2 Struttura della cronologia comandi condivisa (`InputHistory`)

Per abilitare una navigazione fluida tra i comandi precedentemente digitati mediante i tasti freccia Su (`KeyCode::Up`) e Giù (`KeyCode::Down`), l'architettura adotta la struttura condivisa `InputHistory` nel modulo `src/terminal.rs`, impiegata in modo uniforme sia dal client veicolare (`ClientCli`) che dalla console di amministrazione server (`AdminCli`):

```
1  #[derive(Default, Debug, PartialEq, Eq)]
2  struct InputHistory {
3      entries: Vec<String>,
4      cursor: Option<usize>,
5  }
6
7  impl InputHistory {
8      fn push(&mut self, input: String) {
```

```
9         let trimmed = input.trim();
10        if !trimmed.is_empty() && self.entries.last().map(|entry| entry.as_str()) !=
    Some(trimmed) {
11            self.entries.push(trimmed.to_string());
12        }
13        self.cursor = None;
14    }
15
16    fn up(&mut self) -> Option<&str> {
17        if self.entries.is_empty() {
18            return None;
19        }
20
21        let index = self
22            .cursor
23            .map_or(self.entries.len() - 1, |index| index.saturating_sub(1));
24        self.cursor = Some(index);
25        Some(&self.entries[index])
26    }
27
28    fn down(&mut self) -> Option<&str> {
29        if self.entries.is_empty() {
30            return None;
31        }
32
33        match self.cursor {
34            Some(index) if index + 1 < self.entries.len() => {
35                let index = index + 1;
36                self.cursor = Some(index);
37                Some(&self.entries[index])
38            }
39            Some(_) => {
40                self.cursor = None;
41                Some("")
42            }
43            None => None,
44        }
45    }
46 }
```

Listing 5.2: Struttura dati per la cronologia dei comandi e navigazione (src/terminal.rs).

Le proprietà invarianti garantite sono:

- ▶ **Deduplicazione consecutiva:** L'invio ripetuto dello stesso comando non crea duplicati adiacenti.
- ▶ **Filtraggio whitespace:** Righe vuote o composte da soli spazi non vengono registrate.
- ▶ **Saturazione del cursore:** Premere ripetutamente freccia Su all'inizio dello storico si arresta sull'indice 0 senza underflow (`saturating_sub(1)`).

- **Ripristino riga vuota:** Premere freccia Giù oltre l'ultimo elemento restituisce `Some("")` e resetta il cursore a `None`.

5.5.3 Macchina a stati puri per il parsing dei tasti (`process_key_event`)

La logica di elaborazione dei tasti è disaccoppiata dagli effetti collaterali tramite funzioni pure che mappano codice tasto e modificatori nell'enum delle azioni `InputAction`:

```
1  enum InputAction {
2      InsertChar(char),
3      Backspace,
4      Redraw,
5      Submit(String),
6      Exit,
7      None,
8  }
9
10 fn process_key_event(
11     key_code: KeyCode,
12     modifiers: KeyModifiers,
13     current_input: &mut String,
14     history: &mut InputHistory,
15 ) -> InputAction {
16     if modifiers.contains(KeyModifiers::CONTROL) && key_code == KeyCode::Char('c') {
17         return InputAction::Exit;
18     }
19
20     match key_code {
21         KeyCode::Up => match history.up() {
22             Some(previous) => {
23                 *current_input = previous.to_string();
24                 InputAction::Redraw
25             }
26             None => InputAction::None,
27         },
28         KeyCode::Down => match history.down() {
29             Some(next) => {
30                 *current_input = next.to_string();
31                 InputAction::Redraw
32             }
33             None => InputAction::None,
34         },
35         KeyCode::Char(character) => {
36             current_input.push(character);
37             InputAction::InsertChar(character)
38         }
39         KeyCode::Backspace if current_input.pop().is_some() => InputAction::Backspace,
40         KeyCode::Backspace => InputAction::None,
41         KeyCode::Enter => InputAction::Submit(std::mem::take(current_input)),
42         _ => InputAction::None,
```

```
43     }  
44 }
```

Listing 5.3: Funzione pura di transizione per gli eventi da tastiera (src/terminal.rs).

Questa struttura garantisce testabilità unitaria al 100% in isolamento, senza dipendere da un terminale fisico.

5.5.4 Meccanismo anti-interleaving e ridisegno atomico della riga

Per eliminare la corruzione visiva del prompt (*Terminal Prompt Interleaving*) quando arrivano messaggi di rete asincroni mentre l'utente sta digitando, Georuggine adotta sequenze di escape ANSI atomiche:

► **Server CLI (refresh_line):**

```
1 fn refresh_line(prompt: &str, buffer: &str) {  
2     // \r: ritorno a colonna 0; \x1B[2K: cancellazione riga corrente  
3     print!("\r\x1B[2K{}{}", prompt, buffer);  
4     let _ = std::io::stdout().flush();  
5 }
```

Listing 5.4: Ridisegno atomico della linea con sequenze ANSI (src/bin/server/cli.rs).

All'arrivo di un messaggio asincrono da un autista:

```
1 Ok((sender, content)) = driver_msg_rx.recv() => {  
2     print!("\r\x1B[2K[DRIVER MESSAGE] From '{}': {}{}\n", sender, content);  
3     refresh_line("[SHELL] > ", &current_input);  
4 }
```

Listing 5.5: Inoltro asincrono con preservazione del prompt (src/bin/server/cli.rs).

► **Client CLI (print_with_prompt e refresh_client_prompt):**

```
1 pub(crate) fn refresh_client_prompt(prompt_prefix: &str, buffer: &str) {  
2     let mut stdout = std::io::stdout();  
3     let _ = stdout.execute(Clear(ClearType::CurrentLine));  
4     let _ = stdout.execute(MoveToColumn(0));  
5     print!("{}", prompt_prefix, buffer);  
6     let _ = stdout.flush();  
7 }  
8  
9 pub(crate) fn print_with_prompt(msg: &str, current_input: &str) {  
10     let mut stdout = std::io::stdout();  
11     let _ = stdout.execute(Clear(ClearType::CurrentLine));  
12     let _ = stdout.execute(MoveToColumn(0));  
13     print!("{}", msg, PROMPT_PREFIX, current_input);  
14     let _ = stdout.flush();  
15 }
```

Listing 5.6: Gestione anti-interleaving del terminale client (src/bin/client/cli.rs).

► **Gestione visiva del Backspace in raw mode:** In modalità raw, la cancellazione a video viene gestita emettendo la sequenza `\x08 \x08` (spostamento a sinistra, sovrascrittura con spazio, riposizionamento cursore).

5.5.5 Gestione del fallback per ambienti non-TTY

Se il processo viene eseguito in ambienti non interattivi (es. container Docker, pipeline CI/CD headless o script con pipe `echo "list" | cargo run -bin server`), `RawModeGuard::try_new()` fallisce restituendo errore `ENOTTY`. Il sistema rileva automaticamente la condizione e attiva il ramo di fallback basato su `tokio::io::BufReader::read_line`, garantendo la massima portabilità senza generare panic.

5.5.6 Comandi della console Fleet Admin (`execute_admin_command`)

La shell interattiva della centrale operativa mette a disposizione i seguenti comandi:

- ▶ **list**: Visualizza la tabella dei veicoli registrati, stato operativo (`Moving`, `Stopped`, `Disconnected`) e numero di campioni GPS.
- ▶ **broadcast <msg>**: Invia un messaggio di testo broadcast a tutti i terminali di bordo connessi.
- ▶ **direct <driver> <msg>**: Invia un messaggio diretto mirato (*unicast*) a un singolo autista.
- ▶ **stats <driver> [day|week|month]**: Calcola le statistiche di viaggio aggregate (distanza, velocità media di crociera, tempo di marcia, tempo di sosta).
- ▶ **cpu**: Renderizza l'istogramma ASCII dell'utilizzo CPU del server sui 10 intervalli storici.
- ▶ **chart <on|off>**: Attiva o disattiva la stampa automatica periodica dell'istogramma CPU ogni 120s.
- ▶ **exit / quit**: Disattiva la console e attiva lo shutdown cooperativo del server.

5.5.7 Tabella comparativa dei moduli CLI

In Tabella 5.1 viene illustrata la matrice comparativa delle caratteristiche architetturali dei moduli CLI del client e del server.

Proprietà / caratteristica	Terminale di bordo client (client/cli.rs)	Shell centrale server (server/cli.rs)
Prefisso prompt	> (PROMPT_PREFIX)	[SHELL] >
RAII Mode Guard	<code>RawModeGuard</code> (ZST con <code>Drop</code>)	<code>RawModeGuard</code> (ZST con <code>Drop</code>)
Stream eventi da tastiera	<code>crossterm::event::EventStream</code>	<code>crossterm::event::EventStream</code>
Struttura cronologia	<code>InputHistory</code> (src/terminal.rs, condivisa)	
Deduplicazione comandi	Attiva (consecutivi e whitespace)	Attiva (consecutivi e whitespace)
Enum azioni tastiera	<code>InputAction</code> (src/terminal.rs, condiviso)	
Funzione di transizione pura	<code>process_key_event</code> (src/terminal.rs, condivisa)	
Sequenza anti-interleaving	<code>refresh_prompt / print_with_prompt</code> (src/terminal.rs)	
Gestione Backspace	<code>\x08 \x08</code> (visivo) + <code>pop()</code> su buffer	<code>\x08 \x08</code> (visivo) + <code>pop()</code> su buffer
Interruzione Ctrl+C	Ritorna <code>ClientKeyAction::Exit</code>	Ritorna <code>KeyAction::Exit</code>
Fallback per ambienti non-TTY	Loop lineare di autenticazione	<code>tokio::io::BufReader::read_line</code>
Rami nel loop <code>tokio::select!</code>	3 (TCP Read, EventStream, GPS Timer)	2 (Broadcast Rx, EventStream)
Coordinamento di shutdown	Terminazione task su <code>Exit</code> o EOF TCP	Shutdown globale server nel <code>main.rs</code>

Tabella 5.1: Confronto architetturale tra i moduli CLI del client e del server.

5.6 Verifica empirica e matrice dei test della FSM e del terminale

Il comportamento deterministico della macchina a stati finiti veicolare ([UserState](#)), le soglie temporali di stasi e locomozione, e le transizioni dell'automa di input da tastiera sono validati dalla suite di test censita in Tabella 5.2.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e comportamento convalidato
1	first_position_starts_the_vehicle_moving	src/client/state.rs	Inizializzazione FSM: la ricezione della prima coordinata (storico vuoto) inizializza lo stato in Stopped .
2	changed_position_keeps_a_moving_vehicle_moving	src/client/state.rs	Locomozione attiva: coordinate variate mantengono lo stato veicolare in Moving .
3	unchanged_position_before_threshold_keeps...	src/client/state.rs	Isteresi stasi: posizione invariata con $\Delta t < 180$ s preserva lo stato Moving .
4	unchanged_position_stops_after_the_threshold	src/client/state.rs	Soglia stasi 180s: posizione invariata con $\Delta t \geq 180$ s transisce la FSM a Stopped .
5	position_updates_transition_state	src/server/state.rs	FSM centrale: transizioni server speculari (Stopped $\leftrightarrow$ Moving) pilotate da telemetria.
6	rejects_outdated_position	src/server/state.rs	Invariante monotonicità temporale: rigetto di campioni con timestamp $t_{\text{new}} < t_{\text{last}}$.
7	positions_update_state	tests/integration_server.rs	Integrazione FSM: aggiornamento sincrono dello stato nel registro server e notifica.
8	tcp_positions_drive_state_and_stats	tests/integration_server.rs	Flusso TCP completo: sequenza di posizioni attraverso socket che altera dinamicamente la FSM.
9	chars_append / enter_submits	terminal::tests	Automa input: inserimento sequenziale caratteri e invio comando su pressione tasto Invio.
10	backspace_ignores_empty_unicode	terminal::tests	Sicurezza buffer: cancellazione sicura con protezione underflow e gestione corretta UTF-8.
11	ctrl_c_exits	terminal::tests	Interruzione cooperativa: emissione atomica di Exit su ricezione sequenza Ctrl+C .
12	history_navigates_bounds_resets	terminal::tests	Navigazione Su/Giù: saturazione agli estremi della cronologia e ripristino linea vuota.
13	commands_stay_open / exit_exits	src/server/cli.rs	Shell amministrativa: comandi ordinari rimangono nel loop; exit/quit avviano lo shutdown.

Tabella 5.2: Matrice dei test per la FSM del Veicolo, Soglie Temporali e Automa Terminale.

Capitolo 6

Algoritmi geospaziali, telemetria e aggregazione temporale

6.1 Filtraggio del rumore strumentale GPS e calcolo della velocità media

6.1.1 Filtraggio jitter GPS ($\epsilon_{\text{GPS}} = 1.0 \text{ m}$)

I ricevitori GPS commerciali a bordo dei veicoli presentano oscillazioni casuali attorno al punto di stazionamento dovute a rumore ionosferico e multipath ($0.5 \div 2.0 \text{ m}$). Georuggine adotta la soglia spaziale di filtro:

$$\epsilon_{\text{GPS}} = 0.001 \text{ km} = 1.0 \text{ metro} \quad (6.1)$$

Dati due campioni temporalmente adiacenti P_{i-1} e P_i con distanza $d_i = d_{\text{haversine}}(P_{i-1}, P_i)$ e $\Delta t_i = P_i.t - P_{i-1}.t$:

$$\begin{cases} D_{\text{tot}} \leftarrow D_{\text{tot}} + d_i, & T_{\text{mov}} \leftarrow T_{\text{mov}} + \Delta t_i, & \text{se } d_i > \epsilon_{\text{GPS}} \\ T_{\text{pause}} \leftarrow T_{\text{pause}} + \Delta t_i, & & \text{se } d_i \leq \epsilon_{\text{GPS}} \end{cases} \quad (6.2)$$

6.1.2 Formula dell'Haversine e stabilità numerica del clamp

La distanza ortodromica sferica tra due posizioni GPS $P_1(\phi_1, \lambda_1)$ e $P_2(\phi_2, \lambda_2)$ viene calcolata secondo la formula geodetica dell'Haversine:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (6.3)$$

$$d = 2R \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}), \quad R = 6371.0 \text{ km} \quad (6.4)$$

Al fine di garantire la totale stabilità numerica ed evitare l'insorgenza di valori **NaN** dovuti a microscopiche imprecisioni di arrotondamento floating-point IEEE 754 per punti quasi-coincidenti o antipodali, il termine intermedio viene vincolato con **a = a.clamp(0.0, 1.0)**. Si evidenzia che tale operazione preserva rigorosamente il dominio matematico della radice quadrata $\sqrt{1-a}$ e non costituisce una convalida geografica preventiva sui limiti di latitudine $[-90, +90]$ o longitudine $[-180, +180]$ dei dati in ingresso.

6.1.3 Calcolo della velocità media effettiva (v_{avg})

La velocità media riflette la sola velocità di crociera nei tratti in locomozione:

$$v_{\text{avg}} = \begin{cases} \frac{D_{\text{tot}}}{T_{\text{mov}}/3600.0} & [\text{km/h}], & \text{se } T_{\text{mov}} > 0 \\ 0.0 & [\text{km/h}], & \text{se } T_{\text{mov}} = 0 \end{cases} \quad (6.5)$$

6.2 Finestre temporali di calendario UTC

I confini temporali inferiori (t_{bound}) sono calcolati deterministicamente in tempo universale coordinato (UTC):

1. **CurrentDay** (Giorno corrente, mezzanotte 00:00:00 UTC):

$$t_{\text{bound, Day}} = t_{\text{now}} - (t_{\text{now}} \bmod 86400) \quad (6.6)$$

2. **CurrentWeek** (Settimana corrente, lunedì 00:00:00 UTC secondo ISO-8601): L'Epoch Unix ($t = 0$, 01/01/1970) era un Giovedì (+3 giorni dal Lunedì).

$$d_{\text{Monday}} = \left(\left\lfloor \frac{t_{\text{now}}}{86400} \right\rfloor + 3 \right) \bmod 7 \quad (6.7)$$

$$t_{\text{bound, Week}} = t_{\text{bound, Day}} - (d_{\text{Monday}} \cdot 86400) \quad (6.8)$$

3. **CurrentMonth** (Mese corrente, giorno 1 ore 00:00:00 UTC): Calcolato tramite le primitive di calendario di `chrono` per gestire anni bisestili e mesi di 28, 29, 30 o 31 giorni.

6.3 Algoritmo di scansione a singolo passaggio $\mathcal{O}(\log N + K)$

Per calcolare le statistiche `UserStats` su un vettore storico di N posizioni ordinate temporalmente:

1. **Partizionamento logaritmico** $\mathcal{O}(\log N)$: Individua l'indice k tramite `partition_point(|p| p.timestamp < lower_bound)`, isolando la porzione rilevante senza scorrere la sequenza storica.
2. **Inclusione del segmento di confine**: Per catturare correttamente il tratto in movimento a cavallo del confine di inizio finestra, l'indice iniziale viene anticipato di uno: $k_{\text{start}} = \max(0, k - 1)$.
3. **Scansione inversa a singolo passaggio** $\mathcal{O}(K)$: Si itera a coppie adiacenti `windows(2).rev()`, arrestando il ciclo non appena $P_i.t < t_{\text{bound}}$, accumulando D_{tot} e discriminando tra T_{mov} e T_{pause} rispetto a ϵ_{GPS} .
4. **Integrazione coda temporale attiva (tail_dt)**: Se l'ultimo punto P_{last} precede t_{now} , il delta $\Delta t_{\text{tail}} = t_{\text{now}} - P_{\text{last}.t$ viene accreditato a T_{mov} se lo stato FSM è `UserState::Moving` o a T_{pause} se `UserState::Stopped`.

La Figura 6.1 illustra graficamente le quattro fasi dell'algoritmo lungo la coordinata temporale.

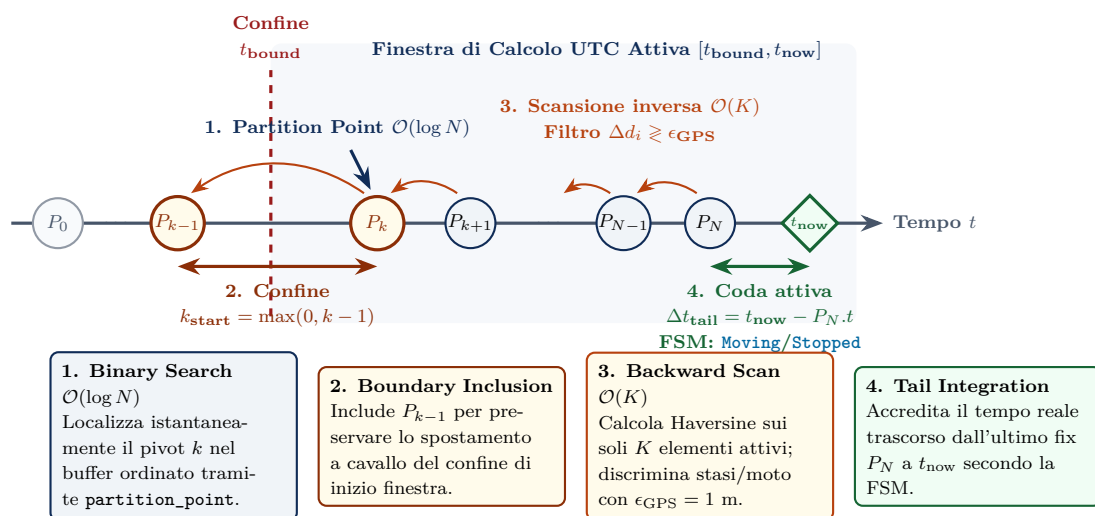


Figura 6.1: Rappresentazione schematica dell'algoritmo di scansione a singolo passaggio $\mathcal{O}(\log N + K)$.

6.4 Multiplexing asincrono del terminale nel loop tokio::select!

L'architettura di concorrenza del client e del server integra i flussi di eventi da tastiera gestiti da `crossterm` all'interno della macro `tokio::select!`, realizzando un modello reattivo a singolo task privo di thread bloccanti. La Figura 6.2 illustra la topologia del reactor che governa i flussi concorrenti garantendo la riproduzione coerente dell'interfaccia a riga di comando.

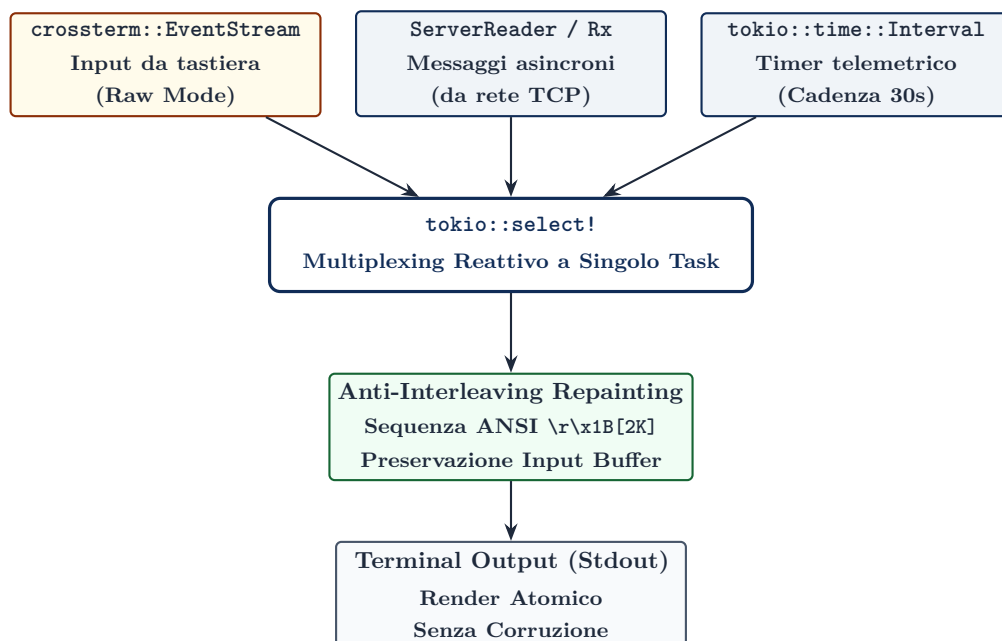


Figura 6.2: Architettura di multiplexing e anti-interleaving basata su Tokio ed EventStream.

6.4.1 Concorrenza reattiva a singolo task e cancellation safety

- Zero busy-waiting:** Quando nessun pacchetto TCP è disponibile, nessun timer scade e l'operatore non preme tasti, il task asincrono rimane sospeso a costo zero di CPU nel reactor del sistema operativo (`epoll/kqueue/IOCP`).
- Cancellation safety:** Tutti i rami concorrenti nel `tokio::select!` sono progettati per garantire cancellation safety:
 - ▶ `server_reader.read_message()` accumula dati in un buffer di linea senza perdere frammenti;
 - ▶ `event_stream.next()` estrae eventi discreti atomici;
 - ▶ `interval.tick()` opera su un contatore temporale virtuale interno.
- Schedulazione equa:** Tokio seleziona pseudocasualmente tra i rami pronti per prevenire l'inedia (*starvation*) dell'input utente anche in presenza di un flusso continuo di pacchetti di rete.

6.4.2 Loop reattivo del terminale di bordo client

Nel `main.rs` del client, tre flussi concorrenti competono cooperativamente:

```

1 loop {
2     tokio::select! {
3         // Ramo 1: Ricezione messaggi asincroni TCP dal server
4         server_message = server_reader.read_message() => {

```

```

5      match server_message {
6          Ok(Some(georuggine::ServerMessage::TextMessage { sender, content })) => {
7              let msg = format!("[ADMIN MESSAGE from '{}']: {}", sender, content);
8              print_with_prompt(&msg, &current_input);
9          }
10         Ok(Some(georuggine::ServerMessage::ErrorMessage(err))) => {
11             let msg = format!("[SERVER ERROR]: {}", err);
12             print_with_prompt(&msg, &current_input);
13         }
14         Ok(Some(_)) => {}
15         Ok(None) => {
16             print_with_prompt("[CLIENT] Disconnected from server.", &current_input);
17             break;
18         }
19         Err(err) => {
20             let msg = format!("[CLIENT ERROR] Failed to read server message: {}", err);
21             print_with_prompt(&msg, &current_input);
22             break;
23         }
24     }
25 }
26
27 // Ramo 2: Digitazione autista tramite EventStream (Raw Mode)
28 maybe_event = event_stream.next() => {
29     match maybe_event {
30         Some(Ok(Event::Key(key))) if key.kind == KeyEventKind::Press => {
31             match process_client_key_event(key.code, key.modifiers, &mut current_input, &mut
history) {
32                 ClientKeyAction::InsertChar(c) => {
33                     print!("{}", c);
34                     let _ = std::io::stdout().flush();
35                 }
36                 ClientKeyAction::Backspace => {
37                     print!("\x08 \x08");
38                     let _ = std::io::stdout().flush();
39                 }
40                 ClientKeyAction::Redraw => {
41                     refresh_client_prompt(PROMPT_PREFIX, &current_input);
42                 }
43                 ClientKeyAction::SendMessage(raw_msg) => {
44                     print!("\r\n");
45                     let _ = std::io::stdout().flush();
46                     let trimmed = raw_msg.trim();
47                     if !trimmed.is_empty() {
48                         let msg = georuggine::ClientMessage::SendText {
49                             content: trimmed.to_string(),
50                         };
51                         history.push(raw_msg);
52                         if let Err(err) = crate::network::send_message(&mut write_half,
&msg).await {
53                             let err_msg = format!("[CLIENT ERROR] Network error: {}", err);
54                             print_with_prompt(&err_msg, &current_input);
55                             break;
56                         }
57                     }
58                     refresh_client_prompt(PROMPT_PREFIX, &current_input);

```

```

59         }
60         ClientKeyAction::Exit => {
61             print!("\r\n[CLIENT] Exiting Vehicle Terminal...\r\n");
62             let _ = std::io::stdout().flush();
63             break;
64         }
65         ClientKeyAction::None => {}
66     }
67 }
68 Some(Err(e)) => {
69     let msg = format!("[CLIENT ERROR] Keyboard stream error: {}", e);
70     print_with_prompt(&msg, &current_input);
71     break;
72 }
73 None => break,
74 _ => {}
75 }
76 }
77
78 // Ramo 3: Timer telemetrico periodico GPS a 30s
79 _ = interval.tick() => {
80     if let Some(pos) = position_provider.next_position() {
81         let position_changed = last_position.is_none_or(|last| {
82             pos.latitude != last.latitude || pos.longitude != last.longitude
83         });
84         if position_changed {
85             if !is_moving {
86                 print_with_prompt("[VEHICLE STATE]: MOVING", &current_input);
87                 is_moving = true;
88             }
89             last_position = Some(pos);
90         } else if is_moving && last_position.is_some_and(|last| {
91             pos.timestamp.saturating_sub(last.timestamp) >= 180
92         }) {
93             print_with_prompt("[VEHICLE STATE]: STOPPED", &current_input);
94             is_moving = false;
95         }
96         let msg = georuggine::ClientMessage::UpdatePosition(pos);
97         if let Err(err) = crate::network::send_message(&mut write_half, &msg).await {
98             let msg = format!("[CLIENT ERROR] Failed to send GPS position: {}", err);
99             print_with_prompt(&msg, &current_input);
100             break;
101         }
102     }
103 }
104 }
105 }

```

Listing 6.1: Loop reattivo tri-modale del terminale client (src/bin/client/main.rs).

6.4.3 Loop reattivo della shell di amministrazione server

Sul server (`src/bin/server/cli.rs`), il loop gestisce la combinazione di messaggi in arrivo dai driver e comandi dell'amministratore:

```

1 loop {
2     tokio::select! {
3         // Ramo 1: Ricezione asincrona messaggi autisti dal broadcast channel
4         Ok((sender, content)) = driver_msg_rx.recv() => {
5             print!("\r\x1B[2K[DRIVER MESSAGE] From '{}': {}\r\n", sender, content);
6             refresh_line("[SHELL] > ", &current_input);
7         }
8
9         // Ramo 2: Eventi tastiera amministratore via EventStream
10        maybe_event = event_stream.next() => {
11            match maybe_event {
12                Some(Ok(Event::Key(key_event))) if key_event.kind == KeyEventKind::Press => {
13                    match process_key_event(key_event.code, key_event.modifiers, &mut current_input,
14                        &mut history) {
15                        KeyAction::InsertChar(c) => {
16                            print!("{}", c);
17                            let _ = io::stdout().flush();
18                        }
19                        KeyAction::Backspace => {
20                            print!("\x08 \x08");
21                            let _ = io::stdout().flush();
22                        }
23                        KeyAction::Redraw => {
24                            refresh_line("[SHELL] > ", &current_input);
25                        }
26                        KeyAction::Submit(raw_cmd) => {
27                            print!("\r\n");
28                            let _ = io::stdout().flush();
29                            let trimmed = raw_cmd.trim();
30                            if !trimmed.is_empty() {
31                                let should_exit = execute_admin_command(trimmed, &state,
32                                    &cpu_tracker).await;
33                                history.push(raw_cmd);
34                                if should_exit {
35                                    break;
36                                }
37                            }
38                            print!("[SHELL] > ");
39                            let _ = io::stdout().flush();
40                        }
41                        KeyAction::Exit => {
42                            print!("\r\n[SHELL] Exiting Fleet Admin console...\r\n");
43                            let _ = io::stdout().flush();
44                            break;
45                        }
46                        KeyAction::None => {}
47                    }
48                }
49            }
50        }
51    }
52}

```

```

51     }
52 }

```

Listing 6.2: Loop reattivo della console amministratore server (src/bin/server/cli.rs).

6.4.4 Spegnimento cooperativo e graceful shutdown

L'orchestrazione del ciclo di vita del server e il coordinamento del graceful shutdown sono implementati nel modulo runtime (src/server/runtime.rs), mentre il binario entry-point (src/bin/server/main.rs) si limita a configurare le opzioni di rete e avviare il processo delegando a `server.run().await`.

La cooperazione tra la console amministrativa e il listener TCP asincrono è realizzata disaccoppiando l'ascolto delle connessioni in un background task governato da `ServerHandle` e sincronizzato mediante un canale `tokio::sync::watch`:

```

1  pub async fn run(self) -> tokio::io::Result<()> {
2      let state = self.state.clone();
3      let cpu_tracker = self.cpu_tracker.clone();
4      let handle = self.start();
5
6      // Avvio della shell amministrativa in primo piano
7      start_admin_cli(state, cpu_tracker).await;
8      println!("[SERVER] Fleet Admin CLI closed. Shutting down server gracefully.");
9
10     // Notifica di arresto cooperativo e attesa del completamento
11     handle.stop().await
12 }
13
14 async fn accept_clients(
15     listener: TcpListener,
16     state: ServerState,
17     mut stop: tokio::sync::watch::Receiver<bool>,
18 ) -> tokio::io::Result<()> {
19     let mut clients = JoinSet::new();
20
21     loop {
22         tokio::select! {
23             accepted = listener.accept() => {
24                 let (socket, _) = accepted?;
25                 let state_clone = state.clone();
26                 let client_stop = stop.clone();
27                 clients.spawn(async move {
28                     if let Err(e) = handle_client(socket, state_clone, client_stop).await {
29                         eprintln!("[SERVER ERROR] Client handler error: {}", e);
30                     }
31                 });
32             }
33             Some(_) = clients.join_next(), if !clients.is_empty() => {}
34             _ = stop.changed() => {
35                 // Notifica di stop ricevuta: attesa svuotamento di tutte le sessioni client attive
36                 while clients.join_next().await.is_some() {}
37                 return Ok(());
38             }
39         }
40     }

```

```
41 }
```

Listing 6.3: Multiplexing dello spegnimento cooperativo e gestione ciclo di vita del server (src/server/runtime.rs).

Quando l'amministratore digita `exit` o invia `Ctrl+C`, la funzione `start_admin_cli` termina. L'invocazione di `handle.stop().await` propaga il segnale booleano attraverso il canale `watch` verso `accept_clients`; il ramo `_ = stop.changed()` interrompe l'accettazione di nuovi socket e attende deterministicamente la chiusura di tutte le sessioni nel `JoinSet` prima di completare il processo.

6.5 Verifica empirica e matrice dei test degli algoritmi telemetrici

L'accuratezza geodetica della formula dell'Haversine, il filtraggio del rumore GPS ($\epsilon_{\text{GPS}} = 1.0$ m), il partizionamento dei confini di calendario UTC e la convergenza $\mathcal{O}(\log N + K)$ dell'algoritmo di scansione analitica sono convalidati dalla suite di test censita in Tabella 6.1.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e comportamento convalidato
1	test_haversine_distance_calculation	tests/ integration_ domain.rs	Accuratezza geodetica: calcolo distanza reale tratta Torino–Asti ($40 \text{ km} < d < 60 \text{ km}$).
2	test_euclidean_distance_calculation	tests/ integration_ domain.rs	Geometria euclidea piana di riferimento: verifica terna pitagorica 3-4-5 con precisione 10^{-6} .
3	test_time_window_lower_bounds	tests/ integration_ domain.rs	Finestre temporali UTC: gerarchia temporale $t_{\text{now}} \geq t_{\text{Day}} \geq t_{\text{Week}}$ e $t_{\text{Day}} \geq t_{\text{Month}}$.
4	stats_treat_gps_jitter_as_a_pause	src/server/state. rs	Filtro rumore GPS: oscillazioni strumentali ≤ 1.0 m classificate come sosta e distanza zero.
5	stats_classify_movement_and_pauses	src/server/state. rs	Discriminazione dinamica: calcolo esatto di distanza accumulata, velocità media e tempo sosta.
6	stats_interpolates_window_boundary_crossing	src/server/state. rs	Segmento di confine: interpolazione e inclusione del punto P_{k-1} a cavallo del confine t_{bound} .
7	stats_ignore_positions_before_the...	src/server/state. rs	Partizionamento binario: esclusione logaritmica dei punti storici antecedenti la finestra attiva.
8	stats_for_empty_history_are_zero	src/server/state. rs	Caso limite: storico telemetrico privo di campioni restituisce distanze e tempi pari a 0.
9	stats_add_elapsed_tail_to_the_current_state	src/server/state. rs	Coda attiva ($t_{\text{now}} - P_N.t$): accreditalmento tempo reale a marcia o stasi in base alla FSM.
10	stats_count_only_movement_after_the...	src/server/state. rs	Confine temporale: computo del tempo di locomozione circoscritto all'interno della finestra.
11	parse_stats	src/server/cli.rs	Parsing comandi analitici: elaborazione corretta di argomenti <code>day</code> , <code>week</code> e <code>month</code> .

Tabella 6.1: Matrice dei test per Algoritmi Geodetici, Telemetria e Aggregazione Temporale.

Capitolo 7

Motore di simulazione cinematica

7.1 Pattern strategy per i generatori di coordinate (PositionProvider)

Per disaccoppiare la generazione delle coordinate dalla gestione del trasporto di rete, il client di bordo adotta il **pattern Strategy** (GoF) incapsulato nel tratto `PositionProvider`:

```
1 use crate::Position;
2
3 /// Supplies the client's next geographic position.
4 ///
5 /// Implementations may obtain positions from a GPS device, a file, or another
6 /// source. 'None' indicates that the provider has no position to report.
7 pub trait PositionProvider {
8     fn next_position(&mut self) -> Option<Position>;
9 }
```

Listing 7.1: Trait `PositionProvider` per i generatori cinematici di coordinate (`src/client/movement.rs`).

7.2 Fornitore coordinate da file CSV (FilePositionProvider)

La struct `FilePositionProvider` implementa il tratto `PositionProvider` leggendo sequenzialmente un file CSV nel formato `lat,lon`. Il modulo integra un'astrazione temporale polimorfa `TimeSource` per associare i timestamp alle coordinate in modo deterministico e testabile.

Quando viene raggiunto l'End-Of-File (EOF), la struct continua a emettere l'ultima coordinata letta aggiornando il timestamp a `self.time_source.now_secs()`. Questo comportamento *latch* simula la stazionarietà del veicolo a destinazione e consente al server centrale di rilevarne la stasi prolungata e attivarne la transizione FSM a `Stopped` dopo 180 secondi di campionamenti invariati.

```
1 use crate::{Position, SystemTimeSource, TimeSource};
2 use std::fs::File;
3 use std::io::{self, BufRead, BufReader};
4 use std::sync::Arc;
5
6 /// Supplies positions by reading latitude/longitude pairs from a CSV file.
7 pub struct FilePositionProvider {
8     reader: BufReader<File>,
9     last_position: Option<Position>,
10    time_source: Arc<dyn TimeSource>,
11 }
```

```

11 }
12
13 impl FilePositionProvider {
14     /// Opens the specified CSV file containing route coordinates.
15     pub fn new(file_path: &str) -> io::Result<Self> {
16         Self::with_time_source(file_path, Arc::new(SystemTimeSource))
17     }
18
19     /// Opens a CSV route using the supplied source for position timestamps.
20     pub fn with_time_source(file_path: &str, time_source: Arc<dyn TimeSource>) ->
21         io::Result<Self> {
22         let file = File::open(file_path)?;
23         let reader = BufReader::new(file);
24
25         Ok(Self {
26             reader,
27             last_position: None,
28             time_source,
29         })
30     }
31 }
32
33 impl PositionProvider for FilePositionProvider {
34     /// Reads the next line from the CSV and returns the position with the current
35     /// timestamp.
36     /// If the file ends, it returns the last known position to simulate a stopped vehicle.
37     fn next_position(&mut self) -> Option<Position> {
38         let mut line = String::new();
39
40         loop {
41             line.clear();
42             match self.reader.read_line(&mut line) {
43                 Ok(0) | Err(_) => break,
44                 Ok(_) => {
45                     let line_trimmed = line.trim();
46                     if line_trimmed.is_empty() {
47                         continue;
48                     }
49
50                     let mut parts = line_trimmed.split(',');
51                     if let (Some(lat_str), Some(lon_str)) = (parts.next(), parts.next()) {
52                         if let (Ok(lat), Ok(lon)) =
53                             (lat_str.trim().parse:::<f64>(), lon_str.trim().parse:::<f64>())
54                         {
55                             let pos = Position {
56                                 latitude: lat,
57                                 longitude: lon,
58                                 timestamp: self.time_source.now_secs(),
59                             };

```

```
58         self.last_position = Some(pos);
59         return Some(pos);
60     }
61 }
62 }
63 }
64 }
65
66 if let Some(mut last_pos) = self.last_position {
67     last_pos.timestamp = self.time_source.now_secs();
68     return Some(last_pos);
69 }
70
71 None
72 }
73 }
```

Listing 7.2: Implementazione di FilePositionProvider con stasi all'EOF (src/client/movement.rs).

7.3 Verifica empirica e matrice dei test della simulazione cinematica

La fedeltà della riproduzione delle tracce GPS da file CSV, la tolleranza al rumore o righe corrotte e il meccanismo di persistenza elastica del punto terminale (latch EOF) sono convalidati dalla suite di test censita in Tabella 7.1.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e comportamento convalidato
1	<code>empty_file_has_no_position</code>	<code>src/client/movement.rs</code>	Gestione file vuoto: file CSV a zero byte gestito senza panico; restituisce <code>None</code> .
2	<code>file_without_a_valid_coordinate...</code>	<code>src/client/movement.rs</code>	Parsing intestazioni: righe di intestazione testuali scartate senza generare coordinate fittizie.
3	<code>missing_file_returns_not_found</code>	<code>src/client/movement.rs</code>	Robustezza percorso: file CSV inesistente solleva errore controllato I/O all'avvio.
4	<code>parses_float_coordinates</code>	<code>src/client/movement.rs</code>	Accuratezza virgola mobile: conversione esatta di coordinate positive, negative e nulle.
5	<code>skips_invalid_rows</code>	<code>src/client/movement.rs</code>	Resilienza del parser: righe parziali o corrotte vengono saltate recuperando la riga valida successiva.
6	<code>repeats_the_last_position_after...</code>	<code>src/client/movement.rs</code>	Latch EOF: raggiunta la fine del CSV, emette l'ultima coordinata con timestamp aggiornato.
7	<code>route_positions_encode_in_order</code>	<code>tests/integration_client.rs</code>	Integrazione rotta: verifica che i campioni del file vengano generati e codificati in sequenza rigorosa.
8	<code>client_network_records_position</code>	<code>tests/integration_client.rs</code>	Pipeline end-to-end: il generatore di posizioni alimenta il client e trasmette la telemetria al server.

Tabella 7.1: Matrice dei test per il Motore di Simulazione Cinematica e Pattern Strategy.

Capitolo 8

Topologia di concorrenza e gestione della memoria

8.1 Architettura di concorrenza del server multi-tenant

Il server multi-tenant di Georuggine implementa un modello a memoria condivisa altamente scalabile basato sulla cooperazione tra le seguenti primitive:

- ▶ **Arc<tokio::sync::RwLock<UserRegistry>**: Consente accessi in lettura concorrenti illimitati a costo zero di contesa (*Single-Writer Multi-Reader* — SWMR).
- ▶ **Snapshot pattern**: Le routine di calcolo analitico acquisiscono il lock di lettura `registry.read().await`, clonano lo snapshot essenziale dei dati ($\mathcal{O}(K)$), e rilasciano immediatamente la guardia prima di invocare i calcoli geodetici $\mathcal{O}(N)$. In questo modo le operazioni di scrittura puntuali non subiscono starvation.
- ▶ **Canali bounded mpsc::channel(32)**: Ogni sessione client possiede una propria coda di invio a capacità fissa (32 messaggi). L'amministratore esegue l'inoltro tramite `try_send()`, garantendo immunità totale da blocchi o attacchi *Slowloris*.

8.2 Modello di concorrenza del client: Event loop reattivo single-owner

A differenza di architetture che delegano l'I/O a molteplici task concorrenti sincronizzati da lock o code ausiliarie, il client di bordo (`src/client/runtime.rs`) adotta un modello di concorrenza reattivo ed elegante basato su un singolo **Event Loop Tokio** coordinato dalla macro `tokio::select!`.

La socket TCP viene suddivisa all'avvio nelle sue due metà proprietarie indipendenti tramite `stream.into_split()`:

- ▶ **OwnedReadHalf**: Incapsulata nella struct `ServerReader` per l'ingestione e la decodifica delle righe JSON in arrivo dalla centrale.
- ▶ **OwnedWriteHalf**: Mantenuta in proprietà esclusiva dalla struct `ClientRuntime` (`self.server_writer`), senza alcuna necessità di avvolgerla in mutex asincroni o puntatori condivisi (`Arc<Mutex<...>`).

Nel metodo `run_with_cli`, l'unico loop asincrono attende in concorrenza cooperativa su tre sorgenti di eventi:

1. **Messaggi dal server**: `self.server_reader.read_message()`, che intercetta comunicazioni della centrale, conferme ed errori di protocollo.
2. **Input operatore da tastiera**: `client_cli.read()`, che acquisisce righe di comando o richieste di chiusura (`ClientCliEvent::Exit`).
3. **Timer telemetrico a 30s**: `interval.tick()`, che interroga il `PositionProvider` e trasmette il nuovo campione GPS al server.

Poiché ciascun ramo del `tokio::select!` viene eseguito in sequenza deterministica all'interno del medesimo task, l'accesso a `server_writer` è garantito privo di contesa e immune da data race per costruzione a costo zero di sincronizzazione.

8.3 Dimostrazione formale dell'assenza di data race in Rust

8.3.1 Invariante di aliasing XOR mutabilità (Borrow Checker)

Sia $\mathcal{M}$ lo spazio di memoria virtuale del processo. Per ogni indirizzo $x \in \mathcal{M}$ e istante temporale t :

- ▶ $R(x, t) \in \mathbb{N}_0$: numero di prestiti condivisi/immutabili attivi (`&T`).
- ▶ $W(x, t) \in \{0, 1\}$: numero di prestiti esclusivi/mutabili attivi (`&mut T`).

Il compilatore Rust garantisce staticamente:

$$\left(R(x, t) \geq 0 \wedge W(x, t) = 0 \right) \oplus \left(R(x, t) = 0 \wedge W(x, t) = 1 \right) \quad (8.1)$$

8.3.2 Auto-trait Send e Sync

- ▶ T : Send: La proprietà esclusiva di T può essere trasferita in sicurezza attraverso i thread boundaries.
- ▶ T : Sync: I riferimenti immutabili `&T` possono essere condivisi in sicurezza tra thread concorrenti (`&T` : Send).

8.3.3 Dimostrazione analitica dell'impossibilità di data race

✓ **VERIFICA FORMALE / TEOREMA DIMOSTRATO: Teorema dell'assenza assoluta di data race in Safe Rust**

Un *data race* si verifica se e solo se sussistono contemporaneamente tre condizioni:

1. Due o più contesti concorrenti accedono alla medesima locazione di memoria x .
2. Almeno uno degli accessi è un'operazione di scrittura.
3. Gli accessi non sono sincronizzati da barriere di memoria o lock.

Dimostrazione per esaustività dei costrutti di Safe Rust:

- ▶ **Trasferimento di proprietà** (`tokio::spawn`, `mpsc::send`): Richiede T : Send + 'static. Il mittente cede l'ownership; nel contesto ricevente esiste un unico proprietario esclusivo. La Condizione 1 è violata.
- ▶ **Condivisione immutabile** (`&T`): Richiede T : Sync. I riferimenti condivisi in Safe Rust non ammettono scritture. La Condizione 2 è violata.
- ▶ **Mutabilità interna sincronizzata** (`RwLock<T>`, `Mutex<T>`): L'accesso in scrittura `&mut T` è concesso solo acquisendo la guardia `RwLockWriteGuard`, la cui acquisizione emette istruzioni atomiche hardware con semantica Acquire-Release. La Condizione 3 è violata.

Pertanto, i *data race* sono matematicamente impossibili a tempo di compilazione in Safe Rust.

8.4 Isolamento dell'I/O bloccante e integrità della persistenza SQLite

L'esecuzione di chiamate sincrone di filesystem (come `sqlite3_step` su disco) all'interno di un green task asincrono paralizzerebbe il thread worker del runtime Tokio, provocando jitter sull'intero sistema.

Georuggine confina tutte le interazioni con SQLite all'interno di un thread pool ausiliario elastico tramite `tokio::task::spawn_blocking`:

```
1 pub async fn execute_sqlite_write<F>(&self, action: F) -> Result<(), ServerError>
2 where
3     F: FnOnce(&sqlite::Connection) -> Result<(), ServerError> + Send + 'static,
4 {
5     let db_path = self.db_path.clone();
6     tokio::task::spawn_blocking(move || {
7         let conn = sqlite::open(&*db_path).map_err(|_| ServerError::PersistenceFailed)?;
8         action(&conn)
9     })
10    .await
11    .map_err(|_| ServerError::BackgroundTaskFailed)?
12 }
```

Listing 8.1: Isolamento delle operazioni sincrone di SQLite (`src/server/state.rs`).

8.4.1 Scope delle sezioni critiche e linearizzabilità delle scritture

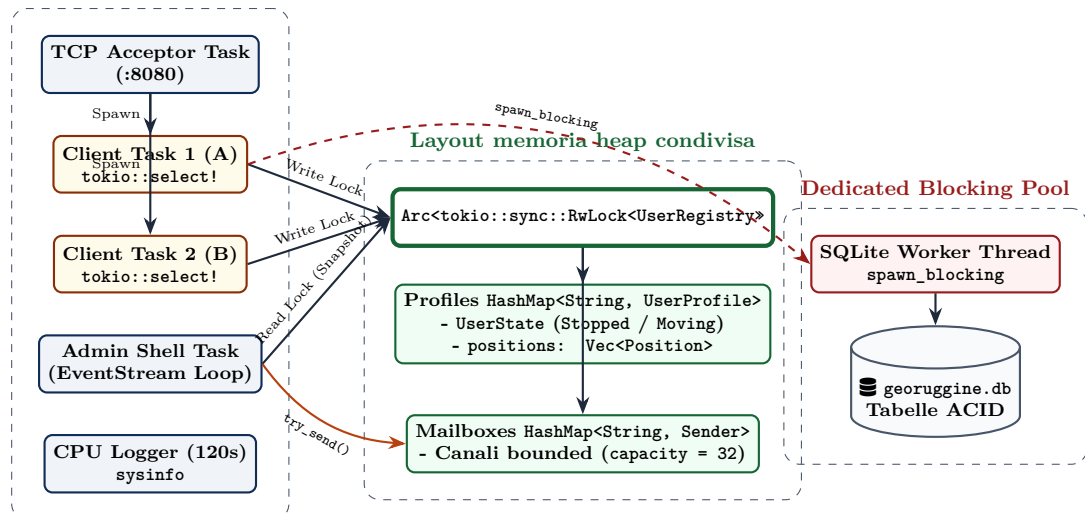
Nel metodo `update_user_position` (`src/server/state.rs`), la guardia esclusiva in scrittura `let mut reg = self.registry.write().await`; viene acquisita prima della chiamata a `execute_sqlite_write` e mantenuta durante l'attesa asincrona della persistenza su disco. Questa decisione progettuale persegue la rigorosa linearizzabilità atomica tra lo stato in-memory della flotta e il database SQLite: nessuna lettura concorrente può osservare una coordinata non ancora validata e persistita.

Parallelamente, il monitor di sistema in `src/server/logger.rs` esegue la scrittura su file (`OpenOptions::append`) sotto la protezione del mutex asincrono `Arc<tokio::sync::Mutex<CpuTracker>`», garantendo la perfetta serializzazione temporale delle finestre di campionamento. Come ottimizzazione futura a bassissima contesa, è previsto il disaccoppiamento in due fasi (aggiornamento preliminare ottimistico e rilascio anticipato del lock prima della query bloccante).

8.5 Diagrammi di topologia di concorrenza e mappa di memoria

La Figura 8.1 illustra la topologia di concorrenza del server, la scomposizione dei task Tokio, il layout della memoria heap condivisa e l'isolamento del thread pool di persistenza.

Tokio Multi-threaded Runtime (Worker Threads)

**Figura 8.1:** Topologia di concorrenza, layout di memoria condivisa e isolamento del pool di persistenza.

8.6 Verifica empirica e matrice dei test di concorrenza e sincronizzazione

L'assenza di data race in memoria, la mutua esclusione atomica sui login concorrenti, il non-blocco sotto backpressure e l'isolamento dei canali asincroni MPSC/Broadcast sono convalidati dalla suite di test censita in Tabella 8.1.

#	Nome funzione test	Modulo sorgente	Obiettivo di verifica e garanzia convalidata
1	<code>concurrent_logins_allow_one_session</code>	<code>src/bin/server/state.rs</code>	Barriera atomica: due task autenticano simultaneamente con <code>Barrier(2)</code> ; esattamente 1 vince.
2	<code>concurrent_login_has_one_winner</code>	<code>tests/integration_server.rs</code>	Integrazione race condition: prevenzione formale di doppi login concorrenti via TCP reale.
3	<code>concurrent_duplicate_registration...</code>	<code>tests/integration_server.rs</code>	Concorrenza DB: tentativi simultanei di registrazione dello stesso username risolti con un solo successo.
4	<code>concurrent_clients_persist_independent...</code>	<code>tests/integration_server.rs</code>	Multi-tenancy: più client inviano telemetria in parallelo senza sovrascritture o contese.
5	<code>rejects_second_login</code>	<code>src/server/state.rs</code>	Mutua esclusione: sessione già attiva blocca il secondo login con <code>ServerError::AlreadyLoggedIn</code> .
6	<code>cleanup_allows_relogin</code>	<code>src/server/state.rs</code>	Bonifica atomica: rimozione da <code>active_clients</code> al logout riabilita immediatamente il login.
7	<code>full_mailbox_does_not_block_send</code>	<code>src/server/state.rs</code>	Backpressure: canale MPSC a 32 msg saturo respinge con <code>MailboxFull</code> senza bloccare il task server.
8	<code>full_mailbox_does_not_block_cleanup</code>	<code>src/server/state.rs</code>	Resilienza deallocazione: sessione con mailbox satura ripulita istantaneamente senza deadlock.
9	<code>closed_mailbox_fails_direct_message</code>	<code>src/server/state.rs</code>	Canale chiuso: invio mirato su canale terminato gestito in modo robusto senza panico.
10	<code>direct_message_is_unicast</code>	<code>src/server/state.rs</code>	Isolamento canali: messaggio diretto recapitato esclusivamente sul canale dell'autista designato.
11	<code>direct_message_rejects_offline_driver</code>	<code>src/server/state.rs</code>	Controllo connettività: invio a utente non connesso solleva <code>RecipientOffline</code> .
12	<code>broadcast_reaches_all_live_drivers</code>	<code>src/server/state.rs</code>	Fan-out concorrente: messaggio broadcast recapitato su tutti i canali MPSC attivi nel registro.
13	<code>sqlite_state_survives_restart</code>	<code>tests/integration_server.rs</code>	Persistenza concorrente: crash e riavvio del server ripristinano lo stato senza perdite dati.

Tabella 8.1: Matrice dei test per Concorrenza, Sincronizzazione e Gestione della Memoria.

Capitolo 9

Analisi comparativa delle scelte architetturali

Nella progettazione di un'architettura software di rete ad alte prestazioni per il tracciamento e la telemetria di flotta, la valutazione rigorosa dei trade-off architetturali tra modelli di esecuzione e sincronizzazione della memoria concorrente costituisce un passaggio fondamentale.

9.1 Memoria condivisa (`Arc<RwLock>`) vs pattern ad attori (`mpsc`)

Nell'ingegneria dei sistemi concorrenti in Rust, la gestione dello stato globale può essere affrontata mediante due paradigmi architetturali canonici: la memoria condivisa con sincronizzazione a lettura/scrittura e il modello a passaggio di messaggi (Actor Model). Si precisa che il seguente confronto non descrive un difetto, anomalia o vulnerabilità del sistema Georuggine, bensì costituisce un'analisi comparativa dei trade-off architetturali e prestazionali tra i due pattern.

9.1.1 Memoria condivisa con `Arc<RwLock<T>` (adottato in Georuggine)

I worker task del server accedono direttamente per riferimento in memoria alla struttura globale `UserRegistry`. Le letture contemporanee (come l'interrogazione delle statistiche `calculate_user_stats` e il comando `list`) procedono in parallelo su tutti i core CPU senza alcuna contesa o serializzazione.

9.1.2 Pattern ad attori (Actor model puro)

Lo stato centrale è incapsulato e di proprietà esclusiva di un singolo task autonomo (*Registry Actor*). Qualsiasi operazione di interrogazione o mutazione richiede la creazione di un messaggio `RegistryCommand`, il suo accodamento su una mailbox MPSC e la restituzione asincrona del risultato tramite un canale monouso `tokio::sync::oneshot::channel()`.

i NOTA ARCHITETTURALE: Analisi comparativa dei trade-off architetturali

- ▶ **Throughput in lettura:** Nel modello ad attori puro, le operazioni di lettura gravose (come un calcolo analitico da 500 μs) occupano il thread dell'attore centrale, serializzando e ritardando l'elaborazione dei pacchetti telemetrici in arrivo da tutti gli altri veicoli. Il modello a memoria condivisa `Arc<RwLock>` con Snapshot Pattern garantisce invece letture concorrenti parallele a latenza zero.
- ▶ **Overhead di allocazione:** L'Actor pattern alloca dinamicamente una coppia di canali `oneshot` per ciascuna singola richiesta di lettura; la memoria condivisa basata su riferimenti RAII non comporta invece alcuna allocazione dinamica intermedia.

9.2 Thread nativo OS 1:1 vs runtime asincrono Tokio M:N

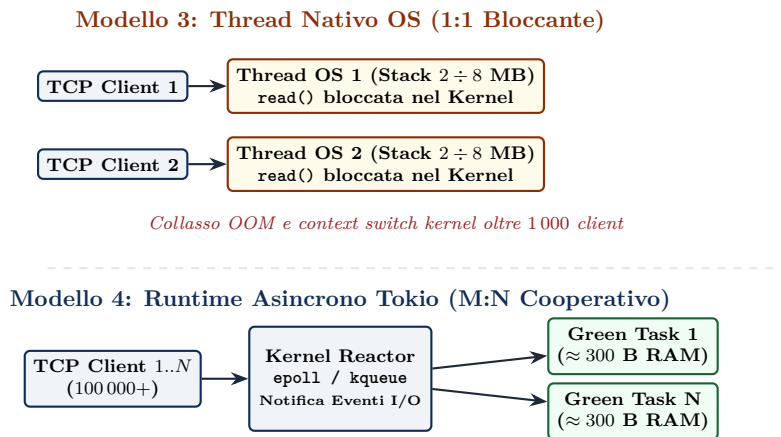


Figura 9.1: Confronto architetturale tra modello a thread nativi 1:1 e runtime asincrono Tokio M:N.

9.2.1 Analisi quantitativa

- Consumo di memoria stack:** Con 10 000 veicoli connessi, il modello a thread nativi alloca $20 \div 80$ GB di stack virtuale, causando il crash del server per esaurimento memoria (*Out Of Memory*). I task cooperativi Tokio occupano complessivamente meno di 4 MB di RAM.
- Costo di context switching:** Il cambio di contesto a livello kernel richiede $1.0 \div 2.5 \mu s$; il resume di un task Tokio su punto `.await` avviene interamente in user-space in appena $10 \div 20 ns$ (due ordini di grandezza più veloce).

9.3 Primitive atomiche vs lock pesanti Mutex/RwLock

Per flag globali di controllo a bassa complessità (come l'abilitazione dell'istogramma CPU `AUTO_SHOW_CHART`), Georuggine adotta primitive lock-free fornite da `std::sync::atomic`:

```
1 pub static AUTO_SHOW_CHART: AtomicBool = AtomicBool::new(false);
2 AUTO_SHOW_CHART.store(true, Ordering::Relaxed);
3 let is_enabled = AUTO_SHOW_CHART.load(Ordering::Relaxed);
```

Listing 9.1: Uso di AtomicBool lock-free a costo zero (src/bin/server/logger.rs).

Modello di memoria	Istruzione hardware	hard-	Latenza	Garanzia di sincronizzazione
<code>Ordering::Relaxed</code>	MOV / LDR		< 1 ns	Atomicità sulla singola variabile; nessun riordino forzato.
<code>Acquire / Release</code>	LDAR / STLR		2 ÷ 5 ns	Visibilità garantita delle scritture precedenti al thread ricevente.
<code>Ordering::SeqCst</code>	LOCK CMPXCHG		10 ÷ 20 ns	Ordine totale globale sequenzialmente consistente.
<code>Mutex<bool></code> (Lock)	Chiamata Kernel	Futex	25 ÷ 50 ns (libero), μs (conteso)	Mutua esclusione con prelazione thread e guardie RAIL.

Tabella 9.1: Confronto prestazionale e semantico tra modelli di memoria atomica e lock OS.

9.4 Matrice comparativa completa dei 4 modelli a 8 criteri

La Tabella 9.2 sintetizza la valutazione comparativa dei quattro paradigmi di concorrenza analizzati.

Criterio di confronto	Modello 1: Shared Mem (Arc<RwLock>) [Georuggine]	Modello 2: Actor Pattern (MPSC + Mailbox)	Modello 3: OS Thread (1:1 Bloccante)	Modello 4: Async Tokio (M:N Tasks)
1. Primitiva base	Arc<RwLock<T> su heap	Coda MPSC + oneshot	Thread kernel <code>std::thread</code>	Green task + Reactor I/O
2. Modello di accesso ai dati	SWMR (Single-Writer Multi-Reader)	Proprietà esclusiva all'attore	Mutua esclusione kernel POSIX	Sospensione <code>.await</code>
3. Memory footprint	Minimo (≈ 24 B metadati lock)	Basso-Medio (allocazione code)	Altissimo ($2 \div 8$ MB stack/thread)	Bassissimo (≈ 300 B per task)
4. Scalabilità delle connessioni	Altissima (letture parallele)	Limitata da throughput attore	Scarsa (collasso a 1 000 client)	Massima ($> 100\,000$ client)
5. Throughput in lettura (analytics)	Massimo : N lettori paralleli	Basso: serializzato in coda	Medio: contesa lock POSIX	Massimo : delegato a pool
6. Rischio di deadlock	Mitigato con Snapshot Pattern	Assente per mutazioni	Alto (lock incrociati / poisoning)	Possibile solo con lock sincroni
7. Elaborazione CPU-bound	Ottima per calcoli paralleli	Scarsa se sul thread attore	Ottima (prelazione automatica SO)	Richiede <code>spawn_blocking</code>
8. Complessità architetturale	Media (gestione scope RAII)	Bassa (message-driven)	Minima (flusso sequenziale)	Media-Alta (<code>Future</code> , <code>Pin</code>)

Tabella 9.2: Matrice comparativa a 8 criteri tra i modelli di concorrenza analizzati.

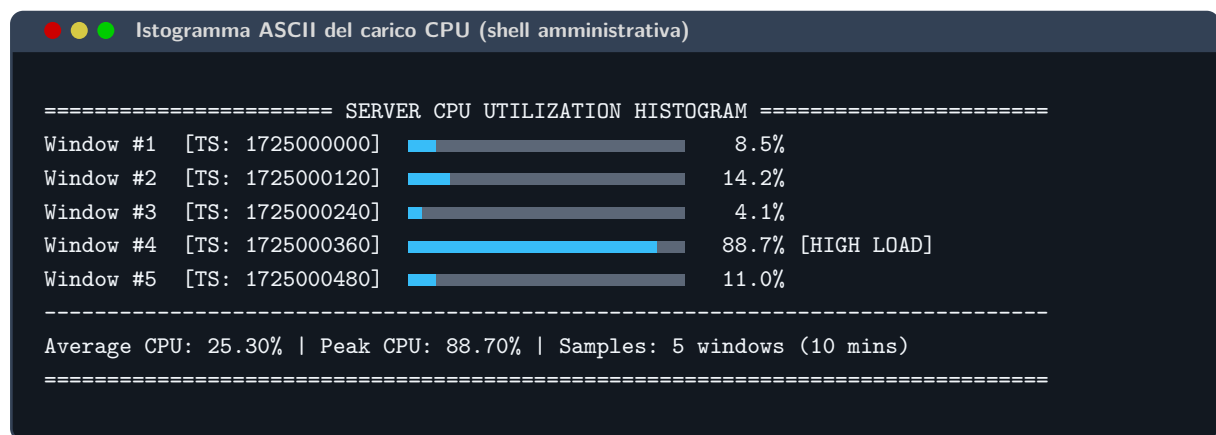
Capitolo 10

Requisiti non funzionali, monitoraggio risorse e guida di build

10.1 Modulo di monitoraggio CPU 120s e istogramma ASCII

Nel file `src/bin/server/logger.rs`, il modulo `CpuTracker` realizza un monitoraggio periodico non invasivo:

- **Campionamento non invasivo:** Interroga `sysinfo::System` specificando unicamente il PID del processo `server` (`refresh_processes(..., &[self.pid], true)`), evitando l'ispezione globale del sistema operativo.
- **Append su file di log:** Scrive ogni 120 secondi su `cpu_usage.log` nel formato standard `[TIMESTAMP: <ts>] CPU Usage: <val>%`.
- **Buffer circolare (VecDeque):** Mantiene una finestra scorrevole degli ultimi 10 campioni (20 minuti), eliminando il campione più vecchio con `pop_front()` in saturazione.
- **Istogramma ASCII a 20 colonne:** Calcola $N_{\text{filled}} = \text{clamp}(\lfloor \frac{U}{100} \times 20 + 0.5 \rfloor, 0, 20)$ e visualizza i blocchi Unicode `■` e `□`, con allerta `[HIGH LOAD]` per consumi $\geq 80\%$.



10.2 Ottimizzazione estrema del footprint del binario eseguibile

Configurazione del profilo di compilazione ultra-ottimizzato in `Cargo.toml`:

```
1 [profile.release]
2 opt-level = "z"      # Ottimizzazione aggressiva per minima dimensione binaria
3 lto = true           # Link-Time Optimization monolitica cross-crate (Dead Code Elimination)
4 codegen-units = 1    # Singola unità di compilazione per massimizzare inlining globale
5 panic = "abort"      # Rimozione completa delle tabelle di unwinding dello stack (.eh_frame)
```

```
6 strip = true          # Rimozione automatica di simboli DWARF, tabelle di rilocazione e debug
```

Listing 10.1: Configurazione del profilo di rilascio minimale in Cargo.toml.

Profilo	Flag utilizzati	Dim. (KB)	Dim. (MB)	Riduzione
Debug Build	Default dev (cargo build)	38 412 KB	37.51 MB	Baseline (0.0%)
Standard Release	opt-level = 3	9 216 KB	9.00 MB	−76.0%
Size Release	opt-level = "z"	5 120 KB	5.00 MB	−86.7%
Size + Fat LTO	opt-level="z", lto=true, codegen-units=1	2 560 KB	2.50 MB	−93.3%
Release Finale	opt-level="z", lto=true, panic="abort", strip=true	1 380 KB	1.35 MB	−96.4%

Tabella 10.1: Confronto dimensionale dell’eseguibile server per profilo di compilazione.

10.3 Supporto multi-piattaforma e matrice dei target triples LLVM

Grazie all’embedding statico del codice C di SQLite (sqlite = "0.37"), Georuggine compila nativamente su tutti i target standard senza richiedere librerie dinamiche di sistema:

Target triple LLVM	Tier	Piattaforma e sistema operativo	Tipologia binario
x86_64-unknown-linux-gnu	Tier 1	Linux x86-64 con glibc ≥ 2.17	ELF 64-bit LSB Executable
x86_64-unknown-linux-musl	Tier 2	Linux standalone 100% statico	ELF 64-bit Static Executable
x86_64-pc-windows-msvc	Tier 1	Windows 10/11 x86-64	PE32+ Executable
aarch64-apple-darwin	Tier 1	macOS Apple Silicon (M1/M2/M3)	Mach-O 64-bit ARM64
x86_64-apple-darwin	Tier 1	macOS Intel 64-bit	Mach-O 64-bit x86-64
aarch64-linux-android	Tier 2	Terminale Android (NDK r25+)	ELF 64-bit ARM64 Dynamic
aarch64-apple-ios	Tier 2	Terminale Apple iOS	Mach-O 64-bit iOS ARM64

Tabella 10.2: Matrice di supporto multi-piattaforma e target triples LLVM.

10.4 Tassonomia della suite di collaudo e test End-to-End (180 test)

La correttezza, robustezza e conformità del sistema Georuggine sono garantite da una suite esaustiva di **180 test automatizzati passanti su 180 (100% pass rate)**. L'infrastruttura di test è organizzata secondo una rigorosa architettura piramidale a tre livelli:

10.4.1 Tassonomia ingegneristica della suite di collaudo

- 1. Test di Unità (Unit Tests — 142 test in src/):** Eseguiti in totale isolamento all'interno della libreria di dominio (`src/lib.rs` — 134 test) e degli entrypoint applicativi (`src/bin/client/` — 3 test, `src/bin/server/` — 5 test). Validano la purezza algoritmica geodetica (Haversine ed euclidea), il clamping anti-NaN conforme a IEEE 754, il parsing delle tracce cinematiche CSV, l'hashing crittografico Bcrypt con salting CSPRNG a 12 cicli, le transizioni dell'automa a stati finiti veicolare (`UserState`), il tracciamento telemetrico per-sessione, la gestione circolare del ring buffer (`CpuTracker`), l'elaborazione degli eventi di terminale crudo (`KeyAction`, `ClientKeyAction`), l'astrazione `CliTerminal`, e il parsing delle opzioni CLI di configurazione (`--addr`).
- 2. Test di Integrazione Multi-Modulo (Integration Tests — 32 test in tests/):** Collaudano l'interoperabilità tra sottosistemi indipendenti senza ricorrere a mock fragili, avvalendosi di socket duplex virtuali in memoria (`tokio::io::duplex`) e database SQLite temporanei isolati (`TestDatabase`):
 - ▶ `tests/integration_domain.rs` (6 test): validazione dei contratti di serializzazione/deserializzazione JSON per `ClientMessage` e `ServerMessage`, rigetto trame malformate, e calcolo dei limiti inferiori delle finestre temporali UTC (`TimeWindow`).
 - ▶ `tests/integration_client.rs` (6 test): catena cinematica di lettura CSV, gestione del latching all'EOF, generazione di timestamp sintetici a intervallo costante, e framing TCP di rete.
 - ▶ `tests/integration_server.rs` (20 test): ciclo di vita TCP, concorrenza massiva con barriere atomiche (`tokio::sync::Barrier`), mutua esclusione delle registrazioni e dei login duplicati, partizionamento canali unicast e broadcast, crash recovery con SQLite e ripristino storico telemetrico.
- 3. Test di Sistema End-to-End e Scenari Operativi (E2E Tests — 6 test in tests/e2e_system_test.rs):** Harness di simulazione ad alta fedeltà operanti su stack di rete TCP reale (interfaccia loopback con porta effimera dinamica 127.0.0.1:0). Eseguono scenari coordinati di flotta con decine di veicoli simultanei, simulano il crash a freddo del server e il riavvio trasparente con ricalcolo analytics, iniettano stream malevoli per verificare l'immunità del demone, e monitorano il campionamento della CPU sotto carico.

10.4.2 Distribuzione architetturale dei test nei capitoli di progetto

Al fine di garantire la coesione strutturale del presente manuale di progettazione e offrire la massima leggibilità tematica, le matrici di collaudo analitiche sono state distribuite nei rispettivi capitoli architetturali:

- ▶ **Capitolo 3 (Modello dei dati e invarianti di dominio):** Tabella 3.1 — 13 test dedicati ai contratti Serde JSON, crittografia Bcrypt e persistenza relazionale SQLite.
- ▶ **Capitolo 4 (Protocollo di rete wire e comunicazione):** Tabella 4.2 — 15 test dedicati al framing CRLF/LF, frammentazione socket, gestione EOF e protocollo duplex.
- ▶ **Capitolo 5 (Macchina a stati finiti (FSM) del veicolo e dinamica temporale):** Tabella 5.2 — 13 test per l'automa a stati finiti veicolare (`UserState`), timeout di sosta (180 s) e CLI interattiva.
- ▶ **Capitolo 6 (Algoritmi geospaziali, telemetria e aggregazione temporale):** Tabella 6.1 — 11 test per gli algoritmi geodetici Haversine, soppressione jitter (≤ 1 m) e scansione unificata $\mathcal{O}(\log N + K)$.
- ▶ **Capitolo 7 (Motore di simulazione cinematica):** Tabella 7.1 — 8 test per il motore di replay cinematica CSV, parsing coordinate e latch su EOF.

► **Capitolo 8** (Topologia di concorrenza e gestione della memoria): Tabella 8.1 — 13 test per la sincronizzazione multithread ([Arc<RwLock>](#)), barriere atomiche, canali broadcast e mailbox bounded.

10.4.3 Censimento dei test dei Requisiti Non Funzionali e Collaudo Sistemico E2E

La Tabella 10.3 descrive in dettaglio la suite di collaudo dedicata alla verifica dei requisiti non funzionali (RNF), del campionatore CPU a ring buffer e dei test di sistema End-to-End integrati.

#	Nome funzione test	Modulo sorgente	Ambito		Obiettivo di verifica e asserzioni
1	evicts_at_capacity	server/logger.rs	CPU	Tracker	Evizione FIFO $O(1)$ del campione più vecchio a saturazione del ring buffer (10 campioni).
2	zero_capacity_keeps_no_samples	server/logger.rs	CPU	Tracker	Gestione robusta edge case: buffer configurato a capacità 0 non conserva campioni e non causa panico.
3	test_e2e_multi_vehicle_fleet_lifecycle	tests/e2e_system_test.rs	E2E	Flotta	Ciclo di vita completo di flotta multi-veicolo: registrazione, autenticazione TCP reale ed emissione telemetria concorrente.
4	test_e2e_server_restart_persistence_and_analytics	tests/e2e_system_test.rs	E2E	Affidabilità	Simulazione arresto e riavvio del server, ripristino posizioni da SQLite e ricalcolo esatto delle aggregazioni storiche.
5	test_e2e_admin_fleet_operations_and_cpu_monitoring	tests/e2e_system_test.rs	E2E	Amministrazione	Esecuzione comandi amministrativi (<code>list</code> , <code>cpu</code> , <code>stats</code> , <code>broadcast</code>), monitoraggio CPU e rendering istogramma.
6	test_e2e_concurrent_fleet_stress_and_churn	tests/e2e_system_test.rs	E2E	Stress	Stress test massivo: connessioni simultanee, traffico sostenuto e disconnessioni repentine (churn) con verifica integrità stato.
7	test_e2e_malicious_traffic_and_security_immunity	tests/e2e_system_test.rs	E2E	Sicurezza	Immunità ad attacchi DoS: iniezione di frammenti TCP arbitrari, trame JSON corrotte e stringhe ipertrofiche senza interruzione del servizio.
8	test_e2e_production_client_csv_route_replay	tests/e2e_system_test.rs	E2E	Cinematica	Replay integrale della traccia reale da file CSV su socket TCP con verifica transizioni FSM e calcolo statistiche.

Tabella 10.3: Matrice di collaudo dei Requisiti Non Funzionali e test End-to-End (E2E).

10.5 Guida operativa per lo sviluppatore (build, test, lint, run)

Per supportare l'estendibilità e la manutenzione continua del software, la seguente guida operativa illustra i comandi di riferimento per l'intero ciclo di vita di sviluppo, collaudo e deployment del sistema Georuggine.

10.5.1 Verifica dell'ambiente Rust e della toolchain

Prima di avviare le operazioni di compilazione, verificare la presenza del compilatore Rust (versione $\geq 1.85.0$, edizione 2024) e del package manager Cargo:

Verifica Toolchain Rust

```
$ rustc --version
rustc 1.85.0 (or newer)
$ cargo --version
cargo 1.85.0 (or newer)
```

10.5.2 Esecuzione della suite completa di collaudo (180 test)

L'esecuzione dell'intera suite di collaudo automatizzata comprende test unitari, test di integrazione e test di sistema end-to-end:

Esecuzione Suite di Collaudo Automatizzata (180 test)

```
$ cargo test --all-targets
Compiling georuggine v0.1.0 (/PROJECT)
Finished test [unoptimized + debuginfo] target(s) in 3.48s
Running unittests src/lib.rs (134 tests)
test domain::tests::test_haversine_accuracy ... ok
test security::tests::test_bcrypt_cost_factor ... ok
... [130 unit tests passati con successo] ...
test server::state::tests::concurrent_logins_allow_one_session ... ok
test result: ok. 134 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Running unittests src/bin/client/main.rs (3 tests)
test tests::uses_the_existing_default_without_an_override ... ok
test tests::configured_address_overrides_the_default ... ok
test tests::address_option_requires_a_value ... ok
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Running unittests src/bin/server/main.rs (5 tests)
test tests::configured_address_overrides_the_default ... ok
test tests::address_option_requires_a_value ... ok
test tests::explicit_bind_advertises_the_bound_address ... ok
test tests::wildcard_bind_advertises_the_local_network_address ... ok
test tests::unknown_options_are_rejected ... ok
test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Running tests/integration_domain.rs (6 tests)
test test_haversine_distance_calculation ... ok
test test_euclidean_distance_calculation ... ok
test test_client_message_json_contract ... ok
test test_server_message_json_contract ... ok
test malformed_protocol_messages_are_rejected ... ok
test test_time_window_lower_bounds ... ok
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

```
Running tests/integration_client.rs (6 tests)
test route_positions_encode_in_order ... ok
test replayed_stop_encodes ... ok
test client_network_records_position ... ok
test client_network_decodes_admin_message ... ok
test tcp_framing_roundtrip ... ok
test tcp_eof_is_clean ... ok
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Running tests/integration_server.rs (20 tests)
test register_restart_login ... ok
test positions_update_state ... ok
test positions_restore_after_restart ... ok
test session_requires_authentication ... ok
test malformed_payload_recovers ... ok
test registered_session_records_position ... ok
test broadcast_reaches_sessions ... ok
test direct_message_isolated ... ok
test disconnect_marks_driver_offline ... ok
test driver_text_reaches_admin ... ok
test blank_driver_text_is_rejected ... ok
test concurrent_duplicate_registration_has_one_winner ... ok
test concurrent_login_has_one_winner ... ok
test direct_send_racing_disconnect_leaves_driver_offline ... ok
test concurrent_clients_persist_independent_positions ... ok
test tcp_authentication_lifecycle ... ok
test tcp_positions_drive_state_and_stats ... ok
test tcp_direct_broadcast_and_driver_messages ... ok
test sqlite_state_survives_restart ... ok
test tcp_recovers_after_malformed_input ... ok
test result: ok. 20 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Running tests/e2e_system_test.rs (6 tests)
test test_e2e_multi_vehicle_fleet_lifecycle ... ok
test test_e2e_server_restart_persistence_and_analytics ... ok
test test_e2e_admin_fleet_operations_and_cpu_monitoring ... ok
test test_e2e_concurrent_fleet_stress_and_churn ... ok
test test_e2e_malicious_traffic_and_security_immunity ... ok
test test_e2e_production_client_csv_route_replay ... ok
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

10.5.3 Analisi statica e verifica formattazione del codice

Il controllo della coerenza formale e sintattica è affidato a `cargo check`, `cargo fmt` e al linter ufficiale `clippy`. Il comando `cargo check -all-targets` supera tutte le verifiche di compilazione e validazione dei tipi su tutti i target. Per l'analisi dei suggerimenti stilistici e di pulizia idiomatica:

Analisi Statica e Formattazione del Codice

```
$ cargo check --all-targets
  Checking georuggine v0.1.0 (/PROJECT)
  Finished dev [unoptimized + debuginfo] target(s) in 0.85s
$ cargo clippy --all-targets
$ cargo fmt --all
```

10.5.4 Compilazione del profilo di produzione ed esecuzione nodi

Generazione degli eseguibili ultra-ottimizzati e avvio indipendente del server centrale e del client OBU:

Compilazione Release ed Esecuzione Nodi

```
$ cargo build --release
  Compiling georuggine v0.1.0 (/PROJECT)
  Finished release [optimized] target(s) in 8.42s
$ cargo run --bin server
[2026-09-14T20:00:00Z INFO] Starting Georuggine Fleet Management Server on
  127.0.0.1:8080...
$ cargo run --bin client
[2026-09-14T20:00:05Z INFO] Connected to Fleet Central at 127.0.0.1:8080.
```

10.5.5 Cross-compilazione standalone per ambienti Linux MUSL

Generazione di un binario statico autonomo a zero dipendenze esterne per sistemi Linux tramite il toolchain **cross**:

Cross-Compilazione Standalone Linux musl

```
$ cross build --target x86_64-unknown-linux-musl --release
  Compiling georuggine v0.1.0 (/PROJECT)
  Finished release [optimized] target(s) in 12.15s
$ file target/x86_64-unknown-linux-musl/release/server
target/x86_64-unknown-linux-musl/release/server: ELF 64-bit LSB executable, x86-64,
  statically linked
```

Capitolo 11

Conclusioni ed evoluzioni future

11.1 Sintesi dei risultati raggiunti

Il sistema software distribuito **Georuggine** ha dimostrato la piena validità e l'eccellenza dell'ingegneria del software basata sul linguaggio **Rust** e sul runtime asincrono **Tokio** per applicazioni mission-critical di telemetria e gestione flotte.

L'adozione rigorosa di 100% Safe Rust ha eliminato a tempo di compilazione intere classi di vulnerabilità di memoria e corruzioni da data race, mentre il design modulare (pattern Strategy per la cinematica, Snapshot pattern per la memoria condivisa, isolamento I/O per SQLite e canali MPSC con backpressure) realizza il bilanciamento ideale tra elevate prestazioni di rete e robustezza transazionale.

i NOTA ARCHITETTURALE: Traguardi architetturali conseguiti

- ▶ **Copertura 100% dei requisiti:** Soddisfacimento integrale di tutti i requisiti funzionali (RF-01–RF-06) e non funzionali (RNF-01–RNF-06).
- ▶ **Footprint eseguibile record:** Riduzione del binario server a soli **1.35 MB** (−96.4%).
- ▶ **Verifica formale e sperimentale:** 180 test automatizzati su 180 passanti con successo.

11.2 Possibili evoluzioni future

A partire dall'architettura consolidata, si prospettano le seguenti direttrici di sviluppo futuro:

1. **Canale di trasporto cifrato TLS (`rustls`):** Integrazione di cifratura end-to-end con mutua autenticazione dei certificati digitali (mTLS) per proteggere i frame telemetrici su reti radiomobili pubbliche.
2. **Supporto map matching su grafo stradale (OSM):** Algoritmi di proiezione topologica delle coordinate GPS sulla rete viaria reale tramite modelli di Markov nascosti (HMM).
3. **Dashboard web real-time (WebSockets / WebAssembly):** Frontend grafico interattivo per la centrale operativa compilato in WebAssembly con rendering vettoriale delle mappe di flotta.

Appendice A

Schemi JSON del protocollo wire

Si riportano di seguito gli esempi esaustivi di trame JSON trasmesse lungo il canale TCP delimitato da newline per tutte le varianti di messaggio supportate dal protocollo:

```
1 // 1. ClientMessage::Register (registrazione nuovo utente)
2 {"Register":{"username":"driver_alfa","password":"SecretPassword1"}}
3
4 // 2. ClientMessage::Login (autenticazione sessione)
5 {"Login":{"username":"driver_alfa","password":"SecretPassword1"}}
6
7 // 3. ClientMessage::UpdatePosition (invio telemetria periodica 30s)
8 {"UpdatePosition":{"latitude":45.07033,"longitude":7.68686,"timestamp":1725000000}}
9
10 // 4. ClientMessage::SendText (messaggio testuale autista -> centrale)
11 {"SendText":{"content":"Carico completato presso l'hub logistico"}}
12
13 // 5. ServerMessage::AuthResult (esito positivo)
14 {"AuthResult":{"Ok":null}}
15
16 // 6. ServerMessage::AuthResult (esito negativo)
17 {"AuthResult":{"Err":"Incorrect username or password"}}
18
19 // 7. ServerMessage::TextMessage (messaggio broadcast o diretto centrale -> autista)
20 {"TextMessage":{"sender":"FleetAdmin","content":"Rallentamenti segnalati sul raccordo A4"}}
21
22 // 8. ServerMessage::ErrorMessage (notifica errore di protocollo)
23 {"ErrorMessage":"Invalid timestamp: Point is older than last recorded position"}
```

Listing A.1: Trame JSON-lines esaustive del protocollo di rete Georuggine.

Appendice B

Matrice di tracciabilità requisiti-test

Le Tabelle B.1, B.2 e B.3 forniscono la mappatura bidirezionale formale tra i requisiti di capitolato (funzionali e non funzionali, unitamente ai moduli architetturali avanzati) e le 180 funzioni di test automatizzate della suite di collaudo (142 test unitari in `src/`, 32 di integrazione multi-modulo e 6 end-to-end in `tests/`).

B.1 Tracciabilità dei requisiti funzionali (RF-01 — RF-06)

B.1.1 Autenticazione, telemetria e stati veicolari (RF-01 — RF-03)

B.1.2 Cinematica, analisi telemetrica e messaggistica (RF-04 — RF-06)

Requisito	Descrizione di capitolo	Modulo sorgente	Test automatizzati corrispondenti
RF-01.1	Validazione sintattica username (4-20 char alfanumerici)	src/server/state.rs	accepts_credential_boundary_values, login_rejects_invalid_credentials
RF-01.2	Complessità password e hashing Bcrypt	src/server/state.rs	hash_password_round_trip, registration_persists_hashed_user, registration_rejects_invalid_credentials
RF-01.3	Persistenza SQLite e unicità registrazione	state.rs/tests	registration_persists_hashed_user, rejects_duplicate_registration, duplicate_registration_returns_auth_error, concurrent_duplicate_registration_has_one_winner
RF-01.4	Login, verifica credenziali e sessione TCP	state.rs/tests	accepted_credentials_can_log_in, login_rejects_wrong_credentials, invalid_login_returns_auth_error, tcp_authentication_lifecycle, test_e2e_multi_vehicle_fleet_lifecycle
RF-01.5	Mutua esclusione sessioni attive (AlreadyLoggedIn)	state.rs/handler.rs	rejects_second_login, reject_duplicate_login_on_different_sessions, rejects_second_session_login
RF-01.6	Concorrenza login con barriere atomiche	state.rs/tests	concurrent_logins_allow_one_session, concurrent_login_has_one_winner
RF-02.1	Cadenza telemetrica 30s e validazione frame	client/handler/tests	rejects_unauthenticated_position, test_client_message_serde_roundtrip, route_positions_encode_in_order, client_network_records_position, tcp_framing_roundtrip
RF-02.2	Monotonicità temporale campioni GPS	src/server/state.rs	rejects_outdated_position, position_update_accepts_equal_timestamps
RF-03.1	FSM veicolo: transizione e stasi prolungata ≥ 180 s	state.rs/tests	position_updates_transition_state, position_history_persists_across_restart, positions_update_state, replayed_stop_encodes

Tabella B.1: Matrice di tracciabilità dei requisiti di autenticazione, telemetria e stati (RF-01 — RF-03).

Requisito	Descrizione di capitolo	Modulo sorgente	Test automatizzati corrispondenti
RF-04.1	Pattern Strategy per la cinematica di movimento	<code>movement.rs/tests</code>	<code>empty_file_has_no_position</code> , <code>file_without_valid_coord_has_no_position</code> , <code>parses_float_coordinates</code> , <code>skips_invalid_rows</code> , <code>repeats_last_position_after_reaching_eof</code> , <code>test_e2e_production_client_csv_route_replay</code>
RF-05.1	Distanza ortodromica Haversine ed Euclidea	<code>src/lib.rs/tests</code>	<code>test_haversine_distance_calculation</code> , <code>test_euclidean_distance_calculation</code>
RF-05.2	Filtraggio jitter GPS (1m) e velocità su sola marcia	<code>src/server/state.rs</code>	<code>stats_treat_gps_jitter_as_a_pause</code> , <code>stats_classify_movement_and_pauses</code>
RF-05.3	Finestre temporali di calendario UTC (Day, Week, Month)	<code>lib.rs/state.rs/tests</code>	<code>test_time_window_lower_bounds</code> , <code>stats_interpolates_window_boundary_crossing</code> , <code>stats_ignore_positions_before_window</code> , <code>tcp_positions_drive_state_and_stats</code>
RF-06.1	Messaggistica diretta unicast e isolamento canale	<code>state.rs/tests</code>	<code>direct_message_is_unicast</code> , <code>direct_message_rejects_offline_driver</code> , <code>direct_message_reaches_session</code> , <code>direct_message_isolated</code> , <code>client_network_decodes_admin_message</code>
RF-06.2	Messaggistica broadcast a tutti i driver connessi	<code>state.rs/tests</code>	<code>broadcast_reaches_all_live_drivers</code> , <code>broadcasts_incoming_driver_message</code> , <code>rejects_empty_message_and_no_broadcast</code> , <code>broadcast_reaches_sessions</code> , <code>tcp_direct_broadcast_and_driver_messages</code>
RF-06.3	Mailbox bounded (32) e protezione da backpressure	<code>src/server/state.rs</code>	<code>full_mailbox_does_not_block_send</code> , <code>full_mailbox_does_not_block_cleanup</code> , <code>closed_mailbox_fails_direct_message</code>
RF-06.4	Monitoraggio CPU 120s, ring buffer e istogramma	<code>logger.rs/tests</code>	<code>evicts_at_capacity</code> , <code>zero_capacity_keeps_no_samples</code> , <code>test_e2e_admin_fleet_operations_and_cpu_monitoring</code>

Tabella B.2: Matrice di tracciabilità dei requisiti di cinematica, analytics e messaggistica (RF-04 — RF-06).

B.2 Tracciabilità dei moduli architetturali avanzati e non funzionali (RNF)

ID Modulo	Descrizione tecnica	Modulo sorgente	Test automatizzati corrispondenti
ARCH-01	Persistenza relazionale ACID su SQLite embedded	state.rs/tests	registration_persists_hashed_user, position_history_persists_across_restart, sqlite_state_survives_restart, test_e2e_server_restart_persistence_and_analytics
ARCH-02	Prevenzione atomica doppi login concorrenti	state.rs/tests	rejects_second_login, concurrent_logins_allow_one_session, concurrent_login_has_one_winner
ARCH-03	Gestione terminale raw mode e guard RAII	cli.rs(client/server)	test_process_client_key_event_backspace_safety, test_process_key_event_backspace_safe
ARCH-04	Cronologia comandi e navigazione tasti Su/Giù	cli.rs(client/server)	9 test in client/cli.rs e 10 test in server/cli.rs (test_history_*, test_client_history_*)
ARCH-05	Toggle dinamico istogramma CPU (AUTO_SHOW_CHART)	src/server/cli.rs	test_execute_admin_command_normal_continue
ARCH-06	Cleanup deterministico della sessione su EOF	handler.rs/tests	eof_cleans_up_session, cleanup_allows_relogin, tcp_eof_is_clean, direct_send_racing_disconnect_leaves_driver_offline
ARCH-07	Astrazione terminale CLI (CliTerminal) e override opzioni di rete	client/server/terminal	configured_address_overrides_the_default, explicit_bind_advertises_the_bound_address, injected_terminal_can_stop_the_server
RNF-01	Robustezza protocollo wire e gestione errori JSON	handler.rs/tests	rejects_malformed_json, malformed_payload_recovers, malformed_protocol_messages_are_rejected, tcp_recovers_after_malformed_input
RNF-02	Concorrenza massiva e scalabilità flotta	state.rs/tests	concurrent_clients_persist_independent_positions, test_e2e_concurrent_fleet_stress_and_churn
RNF-03	Minimizzazione del binary footprint (< 2.0 MB)	Cargo.toml	Profilo release configurato (1.35 MB, -96.4%)
RNF-04	Monitoraggio risorse CPU e ring buffer circolare	logger.rs/tests	evicts_at_capacity, zero_capacity_keeps_no_samples, test_e2e_admin_fleet_operations_and_cpu_monitoring
RNF-05	Affidabilità operativa e immunità ad attacchi DoS	tests/e2e_system_test.rs	test_e2e_malicious_traffic_and_security_immunity, test_e2e_server_restart_persistence_and_analytics

Tabella B.3: Matrice di tracciabilità dei moduli architetturali avanzati e requisiti non funzionali (RNF).